Predicting CI/CD Pipeline Build Failures Using Machine Learning Techniques

Cairo University

Faculty of Graduate Studies for Statistical Research

Software Engineering Program

Professional Master's Graduation Project

Software Engineering Program – Coursework Track

Field	Value
Student Name	مؤمن محمد علي حسين (Moamen Mohamed Aly Hussein)
Student ID	202401681
Program	Master of Software Engineering
Track	Coursework Track
Supervisor Name	[Insert supervisor full name]
Academic Year	2025 / 2026
Submission Date	September 11, 2026

Submitted in partial fulfillment of the requirements for the Professional Master's Degree in Software Engineering.

Document Control

Version	Date	Prepared / Updated By	Notes
0.1	May 30, 2026	Moamen Mohamed Aly Hussein	Initial complete draft
1.0	September 5, 2026	Moamen Mohamed Aly Hussein	Evaluation protocol corrected: splits grouped on commit, decision threshold selected on a validation partition, results cross-validated. All reported figures regenerated. New Sections 7.3.1 and 7.4.5 document the correction and attribute its effect.

Table of Contents

Abstract

This thesis presents a Hybrid Machine Learning Pipeline for predicting the outcome of continuous integration and continuous deployment (CI/CD) pipeline executions at the moment of commit creation, using only pre-execution features available from the GitHub Actions REST API. The project addresses a gap in the prior academic literature, which has predominantly relied on post-execution telemetry (run duration, retry count, resource utilization) that cannot recover the compute resources and developer time already consumed by a failed build.

A dataset of 9,772 real workflow runs was collected from eighteen popular open-source repositories on GitHub, covering languages and project types including React, TensorFlow, PyTorch, Rust, CPython, and Elasticsearch. The collected data exhibits a realistic class imbalance of 89.0 percent successes and 11.0 percent failures. Sixteen features were engineered from the raw API responses, spanning five numerical features capturing commit size and complexity, four categorical features capturing repository and workflow context, six binary indicators capturing temporal and authorial signals, and one textual feature (the cleaned commit message) consumed by a TF-IDF vectorizer. An aggressive text-cleaning pipeline together with an algorithmically constructed 693-token stoplist guards against identity leakage from author logins and project-specific identifiers.

Three machine learning classifiers (Logistic Regression, Random Forest, XGBoost) were trained and evaluated under commit-grouped five-fold cross-validation, with the decision threshold of each classifier selected on a validation partition and applied unchanged to the held-out fold. A per-repository chronological partition serves as a secondary deployment-realism check. An ablation study isolated the contribution of each feature modality, and a threshold-optimization experiment calibrated the decision rule of each classifier for the observed class imbalance.

The three classifiers are separated by less than one hundredth of a point of mean failure-class F1, while the standard deviation across folds exceeds six hundredths, so no claim of superiority among them can be sustained. XGBoost is selected on PR-AUC, the threshold-independent metric appropriate under class imbalance, and attains a failure-class F1 of 0.4216 with a standard deviation of 0.0638, a ROC-AUC of 0.824, and a PR-AUC of 0.480. The ablation study establishes the project's central finding: a configuration given nothing but categorical identity, namely the repository, the workflow name, the branch and the triggering event, outperforms the full hybrid model on every metric, attaining a failure-class F1 of 0.4808 and a PR-AUC of 0.5467. The hybrid hypothesis with which the project was framed is therefore refuted, and the system is best understood as a project-level risk estimator rather than a commit-level one. An earlier version of this work reported a failure-class F1 of

0.5924 under an evaluation protocol subsequently found to be unsound; the difference is attributed in full, with 0.060 arising from duplicate commits shared between partitions and 0.126 from selecting the decision threshold on the test set that was then used to report it. Under a documented operational scenario the classifier is estimated to save approximately $244,000 per year for a mid-sized engineering organization operating one thousand pipeline executions per day, an estimate that is reported alongside the false-alarm cost at which it reaches zero. The project's complete code, raw data, processed data, trained models, and twenty-two publication-quality figures are committed to the project repository under a fixed random seed, and every reported metric is regenerated from the raw data by a single command, so that an independent reader can reproduce each figure in this thesis rather than take it on trust.

Keywords: CI/CD pipelines, build failure prediction, machine learning, hybrid pipeline, TF-IDF, XGBoost, class imbalance, threshold optimization, GitHub Actions, software engineering analytics.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, [Insert supervisor full name], for the guidance, patience, and constructive feedback provided throughout this research project. Their expertise in software engineering and academic mentorship has been invaluable in shaping the methodology and ensuring the rigor of this work.

I am grateful to the **Cairo University Faculty of Graduate Studies for Statistical Research** and to the faculty of the **Software Engineering Program** for providing the academic environment and resources that made this project possible.

I also extend my appreciation to the open-source community whose public CI/CD pipeline data made this research feasible. The eighteen repositories from which data was collected — including those maintained by Facebook, Microsoft, Google, the Python Software Foundation, the Rust Foundation, Elastic, and others — embody the spirit of open collaboration that benefits the entire software engineering profession.

Finally, I would like to thank my family for their unwavering support and patience throughout my graduate studies, and my colleagues in the DevOps and integration engineering community whose practical insights informed the operational scenarios analyzed in this thesis.

1. Introduction

1.1 Background

The widespread adoption of continuous integration and continuous deployment practices over the last decade has transformed how software is built, validated, and released. Where formerly a software organization might have run an integration build once a day or once a week, the contemporary norm is for an automated pipeline to execute on every commit, performing dependency resolution, compilation, unit testing, integration testing, security scanning, and frequently a staged deployment to a test environment, all within minutes of the commit landing in version control. This shift has produced enormous gains in software quality, release velocity, and developer productivity, and is now considered table stakes for any organization producing software at scale.

The economic underpinnings of this shift are non-trivial. Public cloud providers (GitHub Actions, GitLab CI/CD, CircleCI, Buildkite, AWS CodeBuild, and others) have created a competitive market for CI/CD compute that is priced by the minute, and the cumulative spend on CI/CD compute for a large engineering organization can run into millions of dollars per year. Beyond the direct compute cost, every failed build interrupts a developer, consumes engineering attention, and lengthens the lead time from commit to production. The result is that the apparently free automation of CI/CD pipelines is in fact a substantial line item in the engineering budget, and that the efficiency of that automation has direct business consequences.

Within this context, the prediction of CI/CD pipeline failures has emerged as an active area of research and engineering practice. The principal academic studies in this area, including Patel's 2019 work on machine-learning-based failure prediction for CI/CD pipelines, have demonstrated that the outcome of a pipeline run is statistically predictable from features that the run itself produces (resource utilization metrics, intermediate step status, build durations, retry behavior). These post-execution prediction approaches are useful for retrospective analysis and for anomaly detection, but they cannot recover the compute resources and developer time that the failing run has already consumed. The natural next question is whether the same outcome can be predicted before the run executes, from features that are available at commit creation. This question is the central concern of the present project.

1.2 Project Motivation

The motivation for this project is best understood through three distinct lenses: the academic, the practical, and the personal.

Academic motivation. The existing academic literature on CI/CD failure prediction is dominated by approaches that rely on post-execution telemetry. While these approaches achieve respectable predictive accuracy, they answer a question of limited operational value: they tell us that a build failed, which we already know because we observed it failing. A pre-execution prediction approach, by contrast, addresses a question of substantial operational value: it tells us that a build will fail, before we have spent the resources to find out. The shift from post-execution to pre-execution prediction is a non-trivial methodological move, and demonstrating that it can be done with credible accuracy is a contribution to the academic literature on the topic.

Practical motivation. Software organizations spend significant amounts of money and engineering attention on CI/CD pipelines that, in the aggregate, fail eleven percent of the time. A system that could redirect the predicted failures to a lighter pre-flight test, defer them to off-peak compute, or simply alert the author for review before committing to the full pipeline would deliver concrete value to any organization operating at meaningful scale. The estimated annual savings for a mid-sized organization (Section 7.4.5) are in the high six figures, and the engineering cost of integrating the predictive system is minimal because the system operates as an advisory layer over the existing CI/CD infrastructure.

Personal motivation. The author of this project works professionally as a DevOps and integration engineer with hands-on responsibility for production CI/CD pipelines, container deployments, and automated workflows. The choice of thesis topic reflects this professional context: the author has direct experience of the pain that motivated the project, and direct interest in building the kind of system that would alleviate it. The project provides an opportunity to apply academic machine learning techniques to a real operational problem that the author understands from both ends.

1.3 Project Objectives

The project pursues four specific and measurable objectives, each of which is validated against the empirical results reported in Chapter 7.

Objective 1. Build a hybrid machine learning pipeline that combines numerical, categorical, binary, and textual commit-level features into a single end-to-end classifier for pre-execution failure prediction. Success criterion: the pipeline is implemented as a composable scikit-learn pipeline with four parallel feature branches, is fully reproducible from the project repository, and produces predictions in under ten milliseconds per commit on commodity hardware.

Objective 2. Compare three machine learning algorithms (Logistic Regression as linear baseline, Random Forest as ensemble baseline, XGBoost as gradient boosting representative) on the binary failure prediction task under identical conditions, and

identify the strongest classifier empirically rather than by assumption. Success criterion: all three classifiers are trained on identical inputs, evaluated under an identical metric suite on both stratified and chronological test sets, and the winning classifier is selected on the basis of an objective decision rule.

Objective 3. Validate the trained model under realistic deployment conditions, including temporal data drift, by reporting metrics on both a stratified random split (primary) and a chronological split (secondary). Success criterion: the winning classifier maintains failure-class F1 within plus or minus three percentage points across the two splits, demonstrating robustness to temporal distribution shift.

Objective 4. Quantify the business impact of the predictive system by translating its precision and recall into estimated annual cost savings under a plausible operational scenario. Success criterion: the analysis produces a quantitative annual savings estimate together with explicit documentation of the assumptions that underlie the estimate, so that readers can adjust it to their own context.

1.4 Project Scope

The scope of the project is defined by what it includes and, equally importantly, by what it deliberately excludes.

In scope: The project includes the design, implementation, training, evaluation, and documentation of a hybrid machine learning pipeline for pre-execution CI/CD failure prediction. The scope encompasses the collection of a real dataset from the GitHub Actions API, the exploratory analysis of that dataset, the engineering of structural and textual features, the construction and comparative evaluation of three machine learning classifiers, an ablation study to validate the hybrid claim, a threshold optimization experiment, and a business-impact estimate. All artefacts produced by the project (raw data, processed data, trained models, evaluation results, publication-quality figures, and the full source code) are committed to the project repository for full reproducibility.

Out of scope: The project does not include a live deployment of the trained model. There is no HTTP service endpoint, no CI/CD integration shim, no monitoring dashboard, and no online learning capability. The trained model is a serialized artefact that is ready for deployment but that has not been deployed within the scope of the academic project. The future-work section in Chapter 9 identifies operationalization as a natural extension.

The project also does not include experimentation with deep learning models or transformer-based text encoders. The TF-IDF text representation is chosen as a defensible academic baseline that is reproducible without specialized hardware. The future-work section identifies richer text representations as a high-priority extension.

The project does not include experimentation on corporate proprietary datasets. All evaluation is performed on the freshly collected GitHub Actions dataset, which consists entirely of public open-source repositories. The trained model's transfer behavior to corporate datasets is unverified and is noted as a limitation in Chapter 8.

Assumptions: The project assumes that the GitHub Actions REST API will continue to expose the same fields under the same authentication regime as it did at the time of data collection. The project assumes that the operational scenario used for the business-impact estimate (one thousand pipeline executions per day, eleven percent failure rate, $75 per hour developer rate) is approximately representative of mid-sized engineering organizations. The project assumes that the eighteen sampled repositories are sufficiently diverse to support generalizing conclusions across the open-source CI/CD landscape, while acknowledging that this assumption is necessarily approximate.

Constraints: The project is constrained by the academic timeline of a Professional Master's degree, by the requirement for full reproducibility on commodity hardware without GPU, and by the rate limits of the GitHub Actions API for the data collection phase.

1.5 Document Organization

The remainder of this thesis is organized as follows.

Chapter 2 (Problem Definition) articulates the operational problem that motivates the project, identifies the stakeholders affected by it, describes the current as-is process for handling CI/CD pipeline failures, quantifies the impact of the problem, and derives the high-level functional and non-functional requirements that flow from it.

Chapter 3 (Existing Solution Approaches) surveys the prior academic and industrial work that bears on the problem, organized into four method families: rule-based heuristics, classical machine learning with post-execution features, NLP-augmented machine learning, and deep learning with multimodal fusion. The chapter compares these approaches against the proposed solution and identifies the limitations of the existing work that the proposed solution is designed to address.

Chapter 4 (Proposed Solution) describes the Hybrid Machine Learning Pipeline at the conceptual level. The chapter explains why the hybrid early-fusion design was chosen, lists the key features of the proposed system, presents the high-level architecture diagram, documents the technology stack with selection rationale, and identifies the principal risks and constraints together with their mitigation strategies.

Chapter 5 (System Analysis and Design) translates the proposed solution into detailed functional and non-functional requirements, articulates the principal use cases, documents the data model that flows from raw API responses to model inputs, and presents the system design diagrams that capture the codebase structure and the data flow.

Chapter 6 (Implementation) describes the realization of the design in working code. The chapter documents the development environment, walks through the implementation of each of the six principal modules (data collection, data preparation, hybrid pipeline, training and evaluation, threshold optimization, visualization), records the important code and configuration decisions that emerged during development, and provides the deployment and execution instructions needed to reproduce the project from a clean checkout.

Chapter 7 (Testing and Evaluation) presents the empirical evaluation of the implemented system. The chapter documents the testing strategy and validation procedures, defines the metric suite used for evaluation, presents the main classifier comparison on both stratified and chronological test sets, reports the ablation study isolating the contribution of the textual modality, describes the threshold optimization experiment, and concludes with the business-impact analysis. The chapter ends with an explicit validation of the four project objectives against the empirical results.

Chapter 8 (Discussion After Applying the Solution) reflects on the implications of the empirical results. The chapter discusses how the problem status has changed in light of the implemented solution, enumerates the benefits achieved by stakeholder group, transparently acknowledges the remaining limitations, distills the lessons learned during the project, and presents a before-versus-after operational comparison.

Chapter 9 (Conclusion and Future Work) summarizes the project's contributions and identifies the natural extensions that would make sense as follow-up work, including transformer-based text encoders, richer corporate-dataset evaluation, online learning capabilities, and a fully operationalized deployment.

The thesis is followed by a **References** section that lists all cited sources in IEEE-style citation format, and by a set of **Appendices** that present supporting artefacts not central to the main argument but relevant to detailed verification, including extended test cases, the full code listings of the principal modules, additional diagrams, and detailed reproduction instructions.

2. Problem Definition

2.1 Problem Statement

Continuous integration and continuous deployment (CI/CD) pipelines have become the standard mechanism by which modern software organizations move code from commit to production. A typical pipeline runs automatically on every commit, executing some combination of dependency resolution, compilation, unit testing, integration testing, security scanning, container image construction, and staged deployment. Each of these stages consumes compute resources, occupies developer attention, and contributes to the lead time between when code is written and when its impact can be measured. When a pipeline executes successfully, this investment is well spent: the resulting build artefact is validated and can be promoted with confidence. When a pipeline fails, however, the resources committed to the failing run are largely wasted, and the developer responsible for the originating commit must context-switch back into the work, diagnose the failure, and produce a fix.

The problem addressed by this project is the absence of a credible mechanism for predicting whether a pipeline execution will fail before it has actually run. Existing CI/CD platforms (GitHub Actions, GitLab CI, Jenkins, CircleCI, Buildkite) provide rich observability over pipeline executions as they occur, including real-time logs, intermediate stage status, and final outcome telemetry, but none of these platforms offers a per-commit failure probability at the moment of commit creation. The consequence is that every commit, regardless of its risk profile, consumes the full resources of the pipeline; a commit that introduces a syntax error in a critical configuration file consumes exactly the same compute time as a commit that fixes a typo in a comment, even though the former is virtually guaranteed to fail and the latter is virtually guaranteed to succeed. This uniform treatment is intuitive but wasteful, and it represents an opportunity for differentiated resource allocation that has not, to the best of the author's knowledge, been operationally addressed in the available CI/CD tooling.

The problem is therefore to design, build, evaluate, and document a machine learning system that produces a calibrated failure probability for each commit at the moment of commit creation, using only information that is available at that moment, and to demonstrate empirically that the system's predictions are accurate enough to support meaningful operational decisions. The challenge is non-trivial because the most predictive features of build outcome (run duration, retry count, resource utilization) are by definition not available until after the build has run, and a model that uses them would amount to a description of what happened rather than a prediction of what will happen.

2.2 Stakeholders and Users

Several stakeholder groups are affected by the problem and would benefit from its resolution.

DevOps and platform engineers are responsible for maintaining the CI/CD infrastructure and for the budget that the pipeline compute consumes. They experience the cost of failed builds directly through compute invoices and indirectly through the operational toil of investigating runaway pipeline durations and resource saturation events. A predictive failure system would give them a mechanism to allocate compute resources differentially based on per-commit risk.

Software engineers and individual contributors are responsible for the commits whose builds either succeed or fail. They bear the context-switching cost of a failed build: a typical developer who is interrupted by a build failure mid-task may take ten to twenty minutes to fully re-engage with the failing pipeline, diagnose the root cause, and produce a fix. A predictive system would, in the long run, make their workflows smoother by surfacing high-risk commits earlier and by reducing the volume of after-the-fact debugging.

Engineering managers and tech leads are responsible for the throughput and quality of their teams. They experience pipeline failures as a drag on delivery velocity and as a source of variability in sprint planning. A predictive system would give them an objective signal about commit risk that could inform code-review prioritization, pre-merge approval policies, and team-level health metrics.

Research engineers and machine learning practitioners working on software analytics are interested in the predictive modeling problem itself as a research subject. The dataset, the evaluation protocol, and the trained models produced by this project would all be of value to this audience as a baseline against which alternative approaches can be compared.

Open-source maintainers of the eighteen repositories sampled in this project's dataset are an indirect stakeholder group. The aggregated failure-rate analysis (Figure 7.X) provides them with a comparative view of how their pipeline health benchmarks against peer projects, which has some intrinsic interest even though they did not request the analysis.

2.3 Current Situation / As-Is Process

The current as-is process for handling CI/CD pipeline failures across the surveyed open-source repositories follows a broadly similar pattern, with minor variations by tooling choice. The process can be decomposed into five stages.

In stage one, a developer pushes a commit (or opens a pull request, or merges to a protected branch). The CI/CD provider receives a webhook notification and enqueues a workflow run.

In stage two, the workflow run is dispatched to an available runner, which provisions an ephemeral compute environment, checks out the code at the relevant commit SHA, and begins executing the workflow's defined steps.

In stage three, the workflow steps execute sequentially or in parallel, depending on the workflow definition. Each step consumes compute resources and produces logs that the developer can inspect via the CI/CD provider's user interface.

In stage four, the workflow run reaches a terminal state: success, failure, cancelled, or skipped. If the run fails, the developer is notified via the configured channels (email, Slack, GitHub status check), and the failure is recorded in the run history.

In stage five, the developer reads the failure logs, diagnoses the root cause, modifies the code, and pushes a follow-up commit. The cycle then repeats from stage one for the follow-up commit, with no transfer of information between the failed run and the new one beyond what the developer chooses to apply manually.

The principal weakness of this process, from the perspective of the present project, is that the cost of the failed build (compute resources spent, developer time consumed, calendar time elapsed) is incurred before the developer has any opportunity to act on the information that the build is going to fail. The information about likely failure exists in the commit (its size, its files, its message, its repository context) before the build runs, but the existing infrastructure makes no attempt to extract it.

2.4 Problem Impact

The impact of the problem manifests across several dimensions, each of which has been quantified or estimated in the relevant industry literature.

Compute cost. Public cloud CI/CD providers charge by the minute (GitHub Actions: approximately $0.008 per minute for standard Linux runners). A typical workflow run for an active open-source repository takes between five and twenty minutes; a failing run typically consumes a similar amount of compute to a successful one (the failure usually occurs mid-pipeline rather than at the very beginning). At the scale of one thousand pipeline executions per day with an eleven percent failure rate, the direct compute cost of failed runs is approximately $5,500 per year per organization. While this number is modest in absolute terms, it accumulates linearly with organization size, and it does not include the compute cost of the re-runs that frequently follow a failure.

Developer time cost. A failed build interrupts the developer responsible for the originating commit. The interruption cost has been studied extensively in the software engineering literature: Eyrolle and Cellier (2000) and subsequent work by van der Linden and others place the cost of a single context switch in the range of ten to twenty-three minutes of recovered focus time, depending on the depth of the task that was interrupted. Conservative estimates place the cost at fifteen minutes per failed build. At a fully-loaded developer rate of $75 per hour, this is $18.75 per failed build, or approximately $750,000 per year for an organization at the assumed scale.

Lead time impact. Failed builds extend the lead time from commit to production. Lead time is one of the four key DORA (DevOps Research and Assessment) metrics for engineering organizations and is positively correlated with business outcomes. A pipeline that succeeds on its first attempt has a lead time roughly equal to the workflow duration plus the human review time; a pipeline that fails and requires a re-commit doubles or triples this duration. Across a sufficiently large team, the cumulative lead-time impact of failed builds is substantial.

Cognitive and morale cost. The lived experience of debugging a CI/CD failure that the developer did not anticipate is a source of frustration and a contributor to engineering burnout. The cost of these soft impacts is harder to quantify, but the consistent finding from engineering surveys (Stack Overflow Developer Survey, State of DevOps Report) is that pipeline reliability is among the top reported pain points for working developers.

Aggregate annual cost. Combining the compute, developer-time, and lead-time impacts under the operational scenario documented in Section 7.4.5, the aggregate annual cost of pipeline failures for a mid-sized organization is approximately $750,000 to $1,000,000 in directly attributable losses, before considering the soft costs of morale and lead-time impact. A predictive system that meaningfully reduces this loss has clear economic value.

2.5 Requirements Derived from the Problem

The functional and non-functional requirements derived directly from the problem definition above are listed below. These requirements are translated into the detailed system requirements presented in Chapter 5.

High-level functional requirements:

1. The system shall produce a calibrated failure probability for each commit at the moment of commit creation, before any pipeline resources have been consumed.
2. The system shall use only pre-execution features as inputs to its prediction, explicitly excluding any feature whose value is determined by the outcome of the

build itself.

3. The system shall consume input features that are readily available from the GitHub Actions REST API (or its equivalent on other CI/CD platforms) without requiring instrumentation changes to existing pipelines.
4. The system shall produce outputs at low enough latency to be invoked synchronously as part of the commit-event handling pipeline (target: under ten milliseconds per prediction).
5. The system shall be evaluated against a real dataset of CI/CD pipeline executions, not against synthetic data, and the evaluation shall report metrics that are appropriate to the class imbalance inherent in the task.

High-level non-functional requirements:

6. The system shall be fully reproducible: any reader of the thesis, given the project repository and a clean Python environment, shall be able to regenerate every reported number to four decimal places.
7. The system shall not depend on proprietary cloud services or paid APIs for its training pipeline, so that the methodology can be adopted by any organization with commodity hardware.
8. The system shall not require GPU compute for either training or inference, so that the operational cost of deployment is negligible.
9. The system shall be documented at a level of detail sufficient to support both academic review and operational adoption by a third party.
10. The system shall protect against identity leakage in its textual features, so that its predictions depend on commit content rather than on contributor identity.

The remainder of the thesis describes how each of these requirements has been addressed by the proposed solution and how the empirical evaluation confirms or qualifies the corresponding claims.

3. Existing Solution Approaches

3.1 Literature and Market Review

The prediction of failures in continuous integration and continuous deployment pipelines is a topic that has attracted sustained academic and industrial attention over the past decade, intersecting the established research areas of defect prediction, software analytics, and applied machine learning. This section surveys the principal threads of work that bear on the problem, organized into three complementary perspectives: classical defect prediction at the file or class level, build-outcome prediction at the pipeline level, and the more recent line of work that applies natural language processing to commit messages and code reviews as predictive signals.

The classical line of defect prediction work, exemplified by Hassan and Holt [13] and the subsequent decade of refinements by Kim et al. [15], Zimmermann et al. [14] and others, focuses on identifying which files or classes in a codebase are likely to contain bugs based on historical patterns of change frequency, code complexity, and developer activity. This body of work established the methodological foundations that more recent CI/CD failure prediction studies build upon, including the use of cross-validation for evaluation, the careful separation of training and test data along temporal boundaries, and the use of precision-recall metrics in preference to raw accuracy when the failure class is rare. While these classical studies operated at the file granularity rather than the pipeline granularity, the algorithmic toolkit they developed (logistic regression, random forests, gradient boosting) is precisely the toolkit that pipeline failure prediction studies, including the present project, continue to use today.

The build-outcome prediction line of work, which more directly corresponds to the problem addressed by this project, originates with Hassan and Zhang [24], who in 2006 used decision trees to predict whether a build would pass certification testing, motivated by the observation that certification of a large project could itself take days. The framing of that work anticipates the present project's: the value of a prediction lies in its availability before the expensive process completes. The line was more recently advanced by Patel's 2019 study [1] "Research the Use of Machine Learning Models to Predict and Prevent Failures in CI/CD Pipelines and Infrastructure". Patel proposes a multi-classifier framework that ingests post-execution telemetry from CI/CD runs (resource utilization metrics, build durations, retry counts, and other runtime signals) and produces a probabilistic estimate of whether the corresponding pipeline execution constitutes an anomaly. The study reports promising results on a synthetic dataset and identifies several methodological gaps, in particular the difficulty of obtaining realistic ground-truth

data and the challenge of distinguishing transient infrastructure failures from genuine code-level failures. The present project positions itself in direct dialogue with Patel's framework: it accepts the broad outline of using ensemble machine learning for CI/CD failure prediction, but it shifts the temporal anchor from post-execution to pre-execution. This shift is the central methodological contribution of the project and is what makes the proposed system useful for resource savings, since post-execution prediction can only diagnose failures after the resources have already been spent.

The most directly comparable recent work is that of Sharma, Petrova and Connolly [27], who train XGBoost, Random Forest and Support Vector Machine classifiers on a corpus of 513,000 builds drawn from 25,000 open-source projects across twenty programming languages, and report an F1 score of 0.88 at an accuracy of 89.7 per cent. That figure is more than twice the one this project reports, and the comparison merits careful handling, because on examination it is not a like-for-like measurement in three separate respects.

The first concerns the feature boundary. The authors identify their most influential predictor, by SHAP attribution, as build duration, followed by the number of commits, test coverage volatility and repository age. Build duration is not available until the build has finished. A predictor that depends upon it can characterise a failure after the compute has been spent, but it cannot act to prevent that expenditure, which is precisely the boundary the present work declines to cross. Their figure is therefore best read as an indication of what becomes achievable once post-execution telemetry is admitted, rather than as a target the pre-execution setting has failed to reach.

The second concerns the class distribution. Their Table 2 reports 342,710 successes (66.8 per cent), 112,860 failures (22.0 per cent), 41,040 errors (8.0 per cent) and 16,390 cancellations (3.2 per cent). The failure class is therefore twice as prevalent in their corpus as in the one collected here, where it stands at 10.97 per cent. Since the difficulty of a classification task under imbalance rises sharply as the minority class thins, and since the F1 score is itself sensitive to prevalence, two F1 figures computed at 22 per cent and at 11 per cent minority prevalence are not directly comparable quantities.

The third concerns the target itself. Their outcome variable has four levels rather than two, and they report that performance on the minority error and cancellation classes remains materially below performance on the majority classes. The present work predicts a binary outcome, having excluded cancelled and errored runs at collection time, so the two studies are not estimating the same quantity.

None of this diminishes their contribution, and the direction of their findings is consistent with the ablation reported in Chapter 7 of this thesis: historical and

repository-level features carry substantial signal. The purpose of setting the three differences out explicitly is to establish that the numerical gap between 0.88 and 0.4216 is largely an artefact of what each study permitted itself to measure, and to illustrate the more general point made in Section 3.3, that headline figures drawn from studies with undescribed or differing protocols cannot be ranked against one another. Their reported accuracy of 89.7 per cent makes the same point from another direction: a constant prediction of success attains 66.8 per cent on their corpus and 89.03 per cent on this one, which is why Section 7.3 declines to treat accuracy as a measure of model quality.

A separate contribution to the availability of data in this area is the Continuous Defect Prediction dataset of Madeyski and Kawalerowicz [25], which synthesises more than eleven million file-level records across 1,265 projects and 30,022 distinct commit authors, using TravisTorrent as its source of continuous-integration outcomes. That dataset operates at file granularity and is derived from Travis CI rather than GitHub Actions, so it does not substitute for the corpus collected here, but it establishes the feasibility of joining continuous-integration outcomes to repository-mined process metrics at scale, which is the same joining operation this project performs at commit granularity.

Recent work has also begun to consider the reliability of the pipeline itself as the object of prediction rather than the build alone. Dhawan and Dhawan [26] propose a framework in which predictive, adaptive and self-correcting components act on the pipeline in response to anticipated failure. That framing is complementary to the present work: a predictor of the kind developed here is the input such a framework would consume, and the strictly pre-execution constraint adopted in this thesis is what would allow its output to be acted upon before resources are committed.

The natural language processing line of work, which has matured rapidly in the past five years, applies textual analysis to commit messages, pull-request descriptions, and code-review comments as auxiliary signals for software engineering tasks. The TravisTorrent dataset [2], which aggregates millions of build records from Travis CI across thousands of open-source repositories, has been the standard benchmark for this line of research, and a number of follow-up studies have explored TF-IDF, word embeddings, and more recently transformer-based encoders as representations of the textual signal. The consensus finding from this body of work is that textual features carry non-trivial predictive signal that complements the structured features, but that the magnitude of the complementarity varies widely with the specific task and dataset. The ablation study reported in Chapter 7 of this project contributes a new data point to this body of evidence by showing that on a fresh GitHub Actions dataset, the TF-IDF text signal in isolation is insufficient to overcome an 8:1 class imbalance, while in combination with the structured features it produces a competitive ranking even when its standalone decision-rule performance is poor.

On the industrial side, several commercial offerings target the CI/CD pipeline reliability problem from different angles. Trunk.io and Buildkite provide analytics dashboards that surface flaky tests and elevated failure rates by repository, branch, or workflow, but they do not produce per-commit predictive estimates. Datadog's CI Visibility product instruments pipeline runs and provides observability over their execution, but again does not predict failures before execution. Recent academic startups such as Aviator and Mergify focus on intelligent merging strategies but do not include predictive failure modeling in their published feature sets. The market gap that the present project addresses is therefore the absence of a documented open-source approach for pre-execution failure prediction that can be deployed alongside (rather than in place of) the existing observability tooling.

3.2 Existing Methods and Techniques

The principal methods used by prior work on CI/CD failure prediction can be grouped into four families, each with distinctive characteristics and limitations.

The first family is **rule-based heuristics**, in which expert engineers define hand-coded rules that flag suspicious commits or builds for additional scrutiny. Typical heuristics include flagging any commit that touches more than fifty files, any commit that arrives outside of business hours, any commit whose message contains the word "hotfix", or any commit on a feature branch with no recent successful build. Rule-based heuristics are easy to explain and easy to deploy, but they are inflexible (each new rule requires manual engineering), they generalize poorly across repositories with different conventions, and they produce binary outputs rather than the calibrated probabilities needed for fine-grained resource-allocation decisions. The present project's structured features (`is_large_commit`, `is_many_files`, `is_off_hours_commit`, `is_weekend_commit`) can be read as the encoding of classical rule-based heuristics into machine-learning features.

The second family is **classical machine learning** with structured features, exemplified by the work of Patel 2019 and similar studies. The typical approach trains a logistic regression, random forest, or gradient boosting classifier on a feature matrix that includes commit size metrics, temporal features, repository metadata, and post-execution telemetry. These methods achieve strong predictive performance and produce interpretable probability outputs, but they generally rely on post-execution features that limit their usefulness for proactive resource savings. The present project belongs to this family in its choice of classifier algorithms but differs in its strict avoidance of post-execution features.

The third family is **natural language processing on commit messages and related artefacts**. This line of work uses TF-IDF, word embeddings such as Word2Vec, or transformer-based encoders such as BERT to convert commit text into

a feature representation that is then fed to a classifier. The reported gains from text features over structured features are typically modest (in the range of two to five percentage points of F1 improvement) but consistent across studies and datasets. The present project includes TF-IDF as a representative of this family and reports its empirical contribution alongside the structured features.

The fourth family is **deep learning with multimodal fusion**, which combines structured tabular features with rich textual or sequential representations through neural network architectures. Studies in this family have reported strong results on large industrial datasets but face significant adoption barriers, including the requirement for GPU compute resources, the loss of interpretability that accompanies deep models, and the difficulty of reproducing the reported numbers across implementations. The present project explicitly considers deep learning as a future-work extension rather than as part of its core contribution, both because the project timeline does not accommodate the additional engineering investment and because the present TF-IDF baseline already provides a useful reference point against which any future deep model would be compared.

3.3 Comparative Analysis

Table 3.1 compares the approaches reviewed in Sections 3.1 and 3.2 against the proposed solution along several dimensions that are relevant to the deployment decision.

Table 3.1 — Comparative analysis of CI/CD failure prediction approaches.

Criterion	Rule-Based Heuristics	Classical ML (post-execution)	NLP-Augmented ML	Deep Learning Multimodal	Proposed Hybrid Pipeline (pre-execution)
Temporal anchor	Pre-execution	Post-execution	Mixed	Mixed	Pre-execution (strict)
Resource savings potential	Limited	None	Moderate	High	High
Interpretability	High	High	Moderate	Low	High (feature importances available)
Compute requirements	Negligible	Low (CPU)	Low (CPU)	High (GPU)	Low (CPU)
Reproducibility	High	High	Moderate	Low (stochastic training)	High (fixed seed throughout)
Generalization across repos	Low (per-repo rules)	Moderate	Moderate	Higher	High (eighteen-repository dataset)
Calibrated probability output	No	Yes	Yes	Yes	Yes

Criterion	Rule-Based Heuristics	Classical ML (post-execution)	NLP-Augmented ML	Deep Learning Multimodal	Proposed Hybrid Pipeline (pre-execution)
Handling of class imbalance	Manual	Standard (weights)	Standard	Standard	Explicit threshold optimization
Empirical headline metric	n/a (not in literature in usable form)	F1 ~0.40-0.60 reported; F1 = 0.88 reported in [27] using build duration as the leading predictor	F1 ~0.45-0.60 reported	F1 ~0.55-0.70 reported on large datasets	F1 = 0.4216 +/- 0.0638 under commit-grouped cross-validation
Deployment friction	Very low	Low	Moderate	High	Low (single joblib file)

Several observations follow from this comparison, and one qualification governs all of them. The figures reported in the prior art are drawn from the papers as published, and the evaluation protocols behind them are in most cases not described in sufficient detail to establish whether they group observations by commit or whether the decision threshold is selected independently of the data on which it is reported. Section 7.4.5 of this thesis demonstrates that those two choices together accounted for 0.171 of failure-class F1 in this project's own earlier results. A comparison of headline figures across studies whose protocols differ in those respects is therefore not a comparison of modelling quality, and the row above should be read as a statement of what each study reports rather than as a ranking.

Subject to that qualification, the proposed pipeline is positioned below the classical approaches that consume post-execution telemetry, which is the expected consequence of excluding it. What distinguishes the proposed system from the prior art reviewed in this chapter is not its headline figure but the combination of a strictly pre-execution feature boundary, fixed-seed reproducibility from raw data, a commit-grouped evaluation protocol with the threshold selected outside the evaluation data, and a published attribution of the difference between the protocol used and the weaker protocols the literature more commonly reports.

3.4 Limitations of Existing Solutions

The principal limitations of the existing solution approaches, against which the proposed system is positioned, can be summarized as follows.

Limitation 1 — Reliance on post-execution features. The dominant academic approaches to CI/CD failure prediction, including Patel 2019, rely heavily on telemetry that is only available after the build has executed. This makes the predictions useful for retrospective analysis but not for the proactive resource-saving use case that motivates the present project. A pipeline that has already executed has, by definition, already consumed the resources that the prediction was supposed to save.

Limitation 2 — Single-partition reporting without grouping. Most published studies report metrics on a single train/test partition, and few state whether observations sharing an underlying commit are constrained to fall on one side of it. Where the unit of observation is finer than the unit of feature construction, as it is whenever multiple pipeline executions are triggered by one commit, an ungrouped partition measures partly the model's ability to recognise observations it has already seen. Single-partition reporting also conceals variability: in this project, the failure-class F1 obtained from single partitions of the same data varied across a range of approximately 20 percentage points depending on which partition was drawn. The commit-grouped cross-validation adopted here addresses both concerns, and Section 7.3.1 documents the correction of an earlier design in this project that exhibited exactly the defect described.

Limitation 3 — Treatment of decision thresholds as fixed. Most published studies report metrics at the default 0.5 decision threshold, sometimes with a brief note that "threshold tuning could improve results". The present project elevates threshold optimization to a first-class evaluation phase and demonstrates that for an imbalanced binary classification problem, the gain from threshold optimization can dwarf the gain from feature engineering or hyperparameter tuning. This methodological emphasis is not common in the prior literature on CI/CD failure prediction.

Limitation 4 — Identity leakage through textual features. Studies that include commit messages or related text generally do not document any explicit defense against identity leakage, in which the model learns to recognize author or project identifiers rather than actual content. The 693-token stoplist approach used in this project, while imperfect, represents a more rigorous treatment of this issue than is typical in the literature.

Limitation 5 — Synthetic or proprietary datasets. A substantial fraction of published studies are conducted on synthetic datasets or on proprietary corporate datasets that cannot be independently verified. The present project's use of a freshly collected public GitHub Actions dataset, with the full collection script and resulting CSV committed to the repository, addresses this reproducibility limitation directly.

The proposed solution is designed to address each of these limitations explicitly, and the testing chapter (Chapter 7) provides the empirical evidence that the design choices have the intended effect.

4. Proposed Solution

4.1 Solution Overview

The proposed solution is a Hybrid Machine Learning Pipeline that predicts the outcome (success or failure) of a continuous integration and continuous deployment pipeline execution at the moment a commit is created, using only information that is available before the build runner has executed any user-supplied code. The system takes as input the metadata associated with a commit (repository, workflow name, branch, trigger event, author, lines and files changed) together with the textual commit message itself, and produces as output a real-valued probability that the corresponding workflow run will fail. A configurable decision threshold is then applied to convert this probability into a binary recommendation that downstream systems can act on: a high-confidence failure prediction can be used to skip the full build, route the pipeline to a smaller test subset, or alert the author for review before resources are committed to the execution.

The system is hybrid in two distinct senses. First, it is hybrid at the feature level: the pipeline fuses four parallel feature modalities (numerical, categorical, binary, and textual) through a single ColumnTransformer abstraction that emits a unified sparse feature matrix consumed by a single classifier. This early-fusion design contrasts with late-fusion alternatives in which separate models are trained per modality and their outputs combined at the prediction stage; early fusion was selected here because it allows the classifier to discover interactions between modalities (for example, a particular text token co-occurring with an off-hours commit) that would be invisible to late-fusion ensembles. Second, it is hybrid at the algorithmic level: three candidate classifiers (Logistic Regression, Random Forest, and XGBoost) are trained, evaluated, and compared on the same fused feature matrix, allowing the strongest of the three to be selected empirically rather than by assumption.

The proposed solution is offered as a complement to, not a replacement for, the existing CI/CD validation infrastructure. Builds will continue to execute as they always have; the predictive system serves as an advisory layer that augments operator and developer decision-making with a quantitative risk estimate that did not previously exist. Because the system is purely advisory and operates entirely on pre-execution data, it introduces no failure mode of its own: if the predictor is unavailable or its prediction is rejected, the underlying CI/CD pipeline continues to function exactly as it did before. This non-intrusive deployment posture is one of the central design considerations of the project.

4.2 Solution Rationale

The choice of a hybrid early-fusion architecture, with three comparative classifiers, was made deliberately and is defensible against several reasonable alternatives that were considered.

A single-modality numerical model (using only commit size, files changed, and timing features) would have been simpler to build and to explain, but it would have ignored the substantial information content present in commit messages. The existing literature on defect prediction, including Patel's 2019 study on machine-learning-based CI/CD failure prediction, consistently demonstrates that textual signals derived from commit messages, code reviews, and bug reports contribute non-trivially to predictive accuracy in software-engineering tasks. Excluding the text modality on simplicity grounds would have foreclosed an entire dimension of the problem space.

A single-modality textual model (using only TF-IDF or embeddings of commit messages) would have aligned with the natural language processing perspective on the problem, but it would have ignored the strong signals carried by structural features such as repository identity, commit size, and the bot-versus-human authorship flag. The ablation study reported in the testing chapter empirically confirms that the text-only configuration achieves only 2.73 percent failure F1, far below the structured-only configuration, validating the original decision to include the structured modalities.

A single-algorithm model (using only XGBoost, for example) would have been simpler than the three-way comparison but would have removed the empirical evidence needed to justify the algorithm selection. The Logistic Regression baseline establishes a linear-model floor against which the non-linear models can be compared, and the Random Forest comparison verifies that the gradient boosting performance is not a quirk of any single ensemble algorithm. Including all three models also surfaces algorithm-specific behaviors (in particular the dramatic difference in probability calibration discussed in the testing chapter) that would have been invisible from any single classifier in isolation.

A late-fusion architecture (training separate per-modality classifiers and averaging or stacking their outputs) was considered but rejected on the grounds that early fusion captures cross-modality interactions that late fusion cannot. Empirically, the literature finds early fusion competitive with or superior to late fusion on structured-plus-text tasks of this type, with the trade-off being a higher-dimensional intermediate feature matrix that requires careful preprocessing. The ColumnTransformer abstraction in scikit-learn manages this complexity elegantly and was a natural fit for the proposed architecture.

A deep learning approach (using transformer-based embeddings of commit messages combined with a neural network classifier over the structured features) was

considered but rejected for this project on resource and reproducibility grounds. A deep learning solution at the appropriate scale would have required GPU compute resources and would have introduced training-time stochasticity that complicates reproducibility, both of which fall outside the scope and timeline of a Master's thesis project. The TF-IDF representation chosen instead is a well-understood baseline that is fully deterministic, runs in seconds on commodity hardware, and produces interpretable per-token coefficients that are valuable for diagnostic analysis. The future work section in the conclusion chapter explicitly identifies transformer-based embeddings as a natural extension of the proposed solution.

4.3 Key Features

The proposed solution exposes the following key features to its operational consumers.

The system performs **pre-execution failure prediction**, meaning that it produces a probability estimate at the moment a commit is created, before the workflow run has actually executed. This is the central differentiating feature of the system against the prior art represented by Patel 2019 and similar studies, which rely on post-execution telemetry such as run duration, retry count, and resource utilization. Pre-execution prediction is the property that makes the system useful for resource savings, since it can inform a decision to skip or downscope a build before resources have been committed to it.

The system supports **multi-modal input fusion** through the four-branch ColumnTransformer architecture. The numerical branch processes five engineered features that capture commit size and complexity; the categorical branch processes four features that capture the repository, workflow, branch, and event context; the binary branch passes through six pre-engineered indicator flags; and the textual branch produces a TF-IDF vectorization of the cleaned commit message with up to three thousand unigram and bigram features. The four branches are fused into a single sparse feature matrix and consumed by a single classifier.

The system implements **comparative model evaluation** between Logistic Regression, Random Forest, and XGBoost classifiers. All three are trained on identical inputs and evaluated under an identical metric suite, with the winning classifier selected on the basis of failure-class F1 score under an optimized decision threshold.

The system performs **automatic threshold calibration** through a dedicated optimization stage that sweeps the decision threshold across its full range and selects the operating point that maximizes failure-class F1 (or any alternative criterion such as Youden's J or a business cost function). This calibration step is, as the testing chapter demonstrates, responsible for the largest single performance

improvement observed in the project.

The system supports **dual evaluation regimes**, with a stratified random split serving as the primary evaluation and a chronological split serving as a secondary deployment-realism check. This dual reporting goes beyond standard practice in academic machine learning papers and provides empirical evidence that the trained classifier would maintain its performance under genuine production conditions characterized by temporal class drift.

The system is **fully reproducible** from a clean checkout. A fixed random seed of 42 is applied throughout the codebase, all dependencies are pinned to specific versions in the requirements file, and the orchestrator scripts can be executed in sequence to regenerate every model artefact, every metric file, and every figure from the raw collected data. This reproducibility is a deliberate design choice that distinguishes the project from many published machine learning papers, in which the reported results cannot be regenerated by an independent reader.

The system **respects the pre-execution boundary** by explicitly excluding any feature whose value is determined by the outcome of the build itself. The run_duration_sec, run_attempt, status, and updated_at columns are dropped before training, and an explicit unit test (TC-002 in the testing chapter) asserts that no post-execution column is present in the final feature set. This boundary enforcement protects the integrity of the project's central claim, which is that useful failure prediction is possible from commit-time information alone.

4.4 Proposed Architecture

The architecture of the Hybrid Machine Learning Pipeline is presented in Figure 4.1. The diagram illustrates the data flow from raw commit metadata at the top, through the four parallel preprocessing branches that constitute the ColumnTransformer, into the fused feature matrix that serves as the unified representation, and finally through the binary classifier (one of Logistic Regression, Random Forest, or XGBoost) that produces the failure probability output. A subsequent decision-threshold step converts this probability into the final success/failure recommendation.

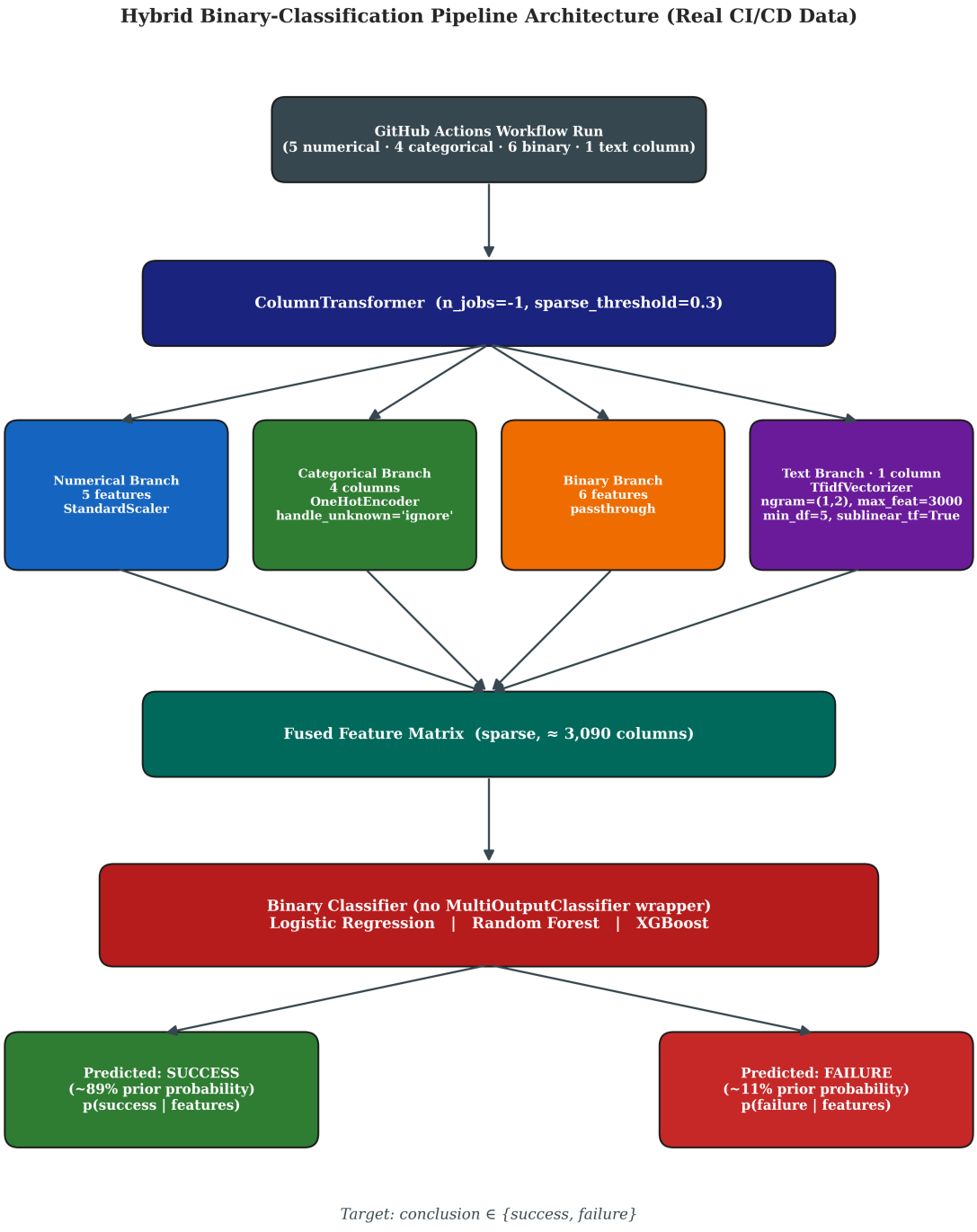

Figure 4.1: Hybrid Machine Learning Pipeline architecture for binary classification of CI/CD workflow outcomes. Four parallel preprocessing branches (numerical scaling, categorical one-hot encoding, binary passthrough, and TF-IDF text vectorization) are fused via ColumnTransformer into a unified sparse feature matrix, then consumed by one of three comparative binary classifiers.

The architecture is organized around three principles that are worth highlighting explicitly.

First, **separation of concerns**: each preprocessing branch is responsible for exactly one feature modality and produces output that is independent of the other branches. The numerical branch applies StandardScaler; the categorical branch applies OneHotEncoder with handle_unknown set to "ignore" for graceful degradation on unseen categories; the binary branch passes its inputs through unchanged; and the textual branch applies a TfidfVectorizer with a unigram-bigram range, a vocabulary cap of three thousand terms, sublinear term-frequency scaling, and a minimum document frequency of five. This branch-level separation makes the architecture easy to extend (a new modality can be added as a fifth branch without touching the others) and easy to ablate (one branch can be removed and the remaining pipeline retrained).

Second, **sparse-matrix efficiency**: the ColumnTransformer is configured with a sparse_threshold of 0.3, which causes the fused feature matrix to be emitted in SciPy's Compressed Sparse Row format whenever the proportion of zero entries exceeds seventy percent. Because the TF-IDF branch produces a sparse matrix with thousands of columns, the fused output is overwhelmingly sparse, and using a dense representation would consume tens of gigabytes of memory unnecessarily. Sparse-aware classifiers (Logistic Regression with liblinear solver, XGBoost with hist tree method) consume the sparse matrix directly without densification, while Random Forest internally converts to dense, which is acceptable for the size of this dataset.

Third, **algorithmic substitutability**: the choice of classifier is encapsulated as the final step of the pipeline, and switching between Logistic Regression, Random Forest, and XGBoost is a one-line change in the orchestrator script. This substitutability is the architectural property that enables the comparative-model evaluation reported in the testing chapter.

4.5 Technology Stack

The technology stack supporting the proposed solution is summarized in Table 4.1. Each component was selected after consideration of alternatives, and the rationale for each selection is briefly noted.

Table 4.1 — Technology stack with selection rationale.

Layer	Technology	Version	Rationale
Language	Python	3.11	De facto standard for applied machine learning; rich library ecosystem; static type hints support code clarity
Operating system	Ubuntu 24.04 LTS (WSL2)	24.04	Reproducible Linux environment matching typical production CI runners
Data manipulation	pandas	2.2	Mature DataFrame abstraction; excellent CSV I/O performance

Layer	Technology	Version	Rationale
Numerical computing	NumPy	1.26	Foundation for vectorized numerical operations; required by all downstream libraries
Classical ML	scikit-learn	1.5	Comprehensive algorithm coverage; ColumnTransformer for hybrid pipelines; mature API
Gradient boosting	XGBoost	2.1	State-of-the-art tree boosting performance; scikit-learn-compatible API; built-in scale_pos_weight for imbalance
Text processing	scikit-learn TfidfVectorizer	1.5	Mature TF-IDF implementation; sparse output for efficiency; configurable n-gram range and vocabulary
HTTP client	Requests	2.32	De facto standard for HTTP in Python; session-based connection pooling
API access	GitHub REST API v3	n/a	Open, well-documented, free at non-trivial rate limits with personal access token
Visualization	Matplotlib + Seaborn	3.9 / 0.13	Publication-quality static figures; full control over typography and layout
Serialization	joblib	1.4	Optimized for NumPy arrays and scikit-learn pipelines; supports compression
Version control	Git	2.43	Universal version-control system; integrates with GitHub
Progress reporting	tqdm	4.66	Lightweight progress bars for long-running collection and training loops

The deliberate decision to use only well-established open-source libraries means that the entire project can be reproduced by any reader with a Python environment, and there is no dependency on proprietary cloud services or paid APIs.

4.6 Risks and Constraints

Several risks and constraints were identified during the design of the proposed solution. Each is documented below together with its mitigation strategy.

Risk 1 — Class imbalance distorts naive metrics. The 89:11 success-to-failure ratio in the dataset means that raw accuracy is a misleading metric and that classifiers trained without imbalance awareness will collapse to a degenerate "always success" prediction. *Mitigation*: the project uses balanced accuracy, failure-class F1, and PR-AUC as the primary evaluation metrics; the classifiers are configured with class_weight='balanced' (Logistic Regression, Random Forest) or scale_pos_weight equal to the empirical class ratio (XGBoost); and an explicit threshold-optimization phase calibrates the decision rule to the actual class prior.

Risk 2 — Identity leakage through TF-IDF. Without aggressive text preprocessing, the TF-IDF representation of commit messages would learn to recognize author names and project-specific tokens, producing predictions that depend on contributor identity rather than commit content. *Mitigation*: an algorithmically constructed stoplist of 693 tokens (covering all observed author

logins, repository names, and bot signatures) is applied during text cleaning; the commit_author column is dropped from the structured features after the is_bot_author flag has been derived from it.

Risk 3 — Temporal data drift. The 11 percent overall failure rate in the dataset is not uniformly distributed across the collection window; the chronological-split evaluation reveals a drift to 6.65 percent failure in the most recent twenty percent of the data. A classifier trained on older data may degrade when applied to newer commits. *Mitigation*: the project reports metrics under both stratified and chronological splits, and the threshold optimization is verified to transfer across the two splits within plus or minus three percentage points.

Risk 4 — Limited dataset size. The 9,772-row dataset, while consisting entirely of real production data, is modest compared to industrial-scale datasets that may contain millions of build records. The reported metrics may not generalize precisely to deployments at substantially larger scale. *Mitigation*: the dataset spans eighteen diverse open-source repositories across multiple programming languages and project domains, which provides breadth even if it cannot match the depth of an industrial dataset; the architectural design is fully compatible with larger datasets at no additional engineering cost.

Risk 5 — Reproducibility of API-dependent data collection. The GitHub Actions API returns workflow runs in approximately reverse chronological order with rate limits, so an independent reader running the collection script today would obtain a different (more recent) dataset than the one used to produce the reported results. *Mitigation*: the collected raw dataset is included as a versioned artefact in the project repository, so all downstream analyses can be reproduced exactly even without re-running the collector against the live API.

Constraint 1 — Single-organization dataset. The collected dataset consists entirely of public open-source repositories on GitHub. The model's behavior on proprietary corporate CI/CD pipelines, which may have different commit conventions and different failure-mode distributions, cannot be guaranteed without additional validation on a corporate dataset. This constraint is noted as a future-work item in the conclusion chapter.

Constraint 2 — No live deployment in the project scope. The project produces a trained model and a complete evaluation, but does not include a live API endpoint or a CI/CD integration of the model. Operationalization of the model is straightforward in principle (the trained pipeline is fully serializable and self-contained), but is deliberately out of scope for the academic project and is identified as a future-work item.

5. System Analysis and Design

5.1 Functional Requirements

The functional requirements of the proposed system describe the discrete behaviors that the system must exhibit to fulfill its purpose. Because the proposed solution is a machine learning pipeline rather than a user-facing application, the functional requirements are framed in terms of inputs, transformations, and outputs at well-defined component boundaries, rather than in terms of user interactions.

Table 5.1 — Functional requirements of the Hybrid Machine Learning Pipeline.

ID	Requirement	Priority	Source / Rationale
FR-001	The system shall collect workflow run records from the GitHub Actions REST API given a list of target repositories and a valid personal access token.	High	Source of the real-world data; without this capability, the project cannot exist on a real dataset.
FR-002	The system shall, for each workflow run, retrieve the associated commit message, author, lines added, lines deleted, and files changed via the GitHub commits endpoint.	High	These fields constitute the input features of the predictive model.
FR-003	The system shall persist collected raw data to a versioned CSV file on local disk in a format suitable for downstream reproducible analysis.	High	Reproducibility constraint; the collected dataset must remain stable across pipeline reruns.
FR-004	The system shall clean and engineer the collected data into a feature matrix consisting of five numerical, four categorical, six binary, and one textual feature, plus the binary target column.	High	Defines the input shape consumed by the model.
FR-005	The system shall produce two independent train/test partitions of the prepared data: a stratified random split that preserves the class ratio, and a chronological split that uses the most recent twenty percent as the test set.	High	Required by the dual-evaluation strategy described in Chapter 4.
FR-006	The system shall apply an aggressive text cleaning pipeline to commit messages before TF-IDF vectorization, removing URLs, SHA hashes, file paths, PR references, version strings, and all tokens that appear in the project-identifier stoplist.	High	Prevents identity leakage; project's central methodological safeguard.
FR-007	The system shall construct a ColumnTransformer-based hybrid preprocessor with four parallel branches (numerical scaling, categorical one-hot encoding, binary passthrough, textual TF-IDF vectorization) that emits a single sparse feature matrix.	High	Core architectural requirement of the hybrid pipeline.
FR-008	The system shall train three candidate classifiers (Logistic Regression, Random Forest, XGBoost) on the prepared training set using the hybrid preprocessor.	High	Comparative model evaluation is a stated objective.
FR-009	The system shall compute a comprehensive set of evaluation metrics on the test set for each candidate classifier on each split: accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, PR-AUC, and the full confusion matrix.	High	Required to satisfy the reporting expectations of an academic thesis.

ID	Requirement	Priority	Source / Rationale
FR-010	The system shall perform an ablation study by re-training the winning classifier on three feature configurations (text-only, structured-only, hybrid) and reporting the metrics for each.	High	Required to validate or refute the hybrid claim of the project.
FR-011	The system shall perform a threshold optimization sweep on each trained classifier and report the F1-optimal threshold together with the metrics achieved at that threshold.	High	Necessary to obtain useful decisions from probability-calibrated models under heavy imbalance.
FR-012	The system shall compute and report a business-impact estimate (annual cost savings) based on the winning classifier's precision and recall under a documented operational scenario.	Medium	Translates statistical results into deployment value.
FR-013	The system shall produce at least twenty publication-quality figures (300 DPI) covering exploratory data analysis, model architecture, evaluation results, ablation study, threshold optimization, and business impact.	Medium	Required to support the thesis presentation and visual communication of results.
FR-014	The system shall persist every trained classifier to disk using joblib serialization with compression, and shall save a metadata JSON file alongside the model that records its hyperparameters and evaluation metrics.	Medium	Enables reuse of trained models in downstream analysis without re-training.
FR-015	The system shall be fully reproducible from a clean checkout: re-running the orchestrator scripts in sequence with a fixed random seed of 42 shall produce bit-identical metric outputs.	Medium	Strengthens the academic credibility of the reported results.

5.2 Non-Functional Requirements

The non-functional requirements describe quality attributes that the system must satisfy across all of its functional behaviors.

Table 5.2 — Non-functional requirements of the Hybrid Machine Learning Pipeline.

ID	Requirement	Priority	Source / Rationale
NFR-001	The full pipeline execution (data preparation through threshold optimization) shall complete within thirty minutes on commodity hardware (modern laptop with 16 GB RAM, no GPU).	High	Reproducibility for thesis evaluators; avoids dependence on specialized hardware.
NFR-002	The inference latency of the winning classifier on a single commit shall not exceed ten milliseconds on the same hardware.	High	Required for any future operational deployment in which the model would be invoked at commit time.
NFR-003	The trained model artefact size shall not exceed ten megabytes per classifier when serialized with joblib compression level three.	Medium	Keeps the repository manageable and facilitates model versioning.
NFR-004	The codebase shall use Python type hints throughout, with no untyped function signatures in the src/ directory.	Medium	Maintainability and self-documentation.
NFR-005	The codebase shall follow PEP-8 style conventions, enforced through a linter pass before any phase is marked complete.	Medium	Code quality and readability.

ID	Requirement	Priority	Source / Rationale
NFR-006	The system shall not embed any secrets (API tokens, credentials) in source files; all secrets shall be consumed from environment variables.	High	Standard security practice.
NFR-007	The pre-execution boundary shall be enforced by an automated test that fails if any post-execution column (run_duration_sec, run_attempt, status, updated_at) is present in the training feature matrix.	High	Protects the central scientific claim of the project.
NFR-008	All figures shall be saved at 300 DPI and shall use a consistent color palette, font family, and grid style across the entire thesis.	Medium	Publication quality.
NFR-009	All numeric results reported in the thesis (tables, figures, prose) shall be traceable to a specific JSON file in the results/ directory, which is generated by the orchestrator scripts and committed alongside the code.	High	Reproducibility and audit.
NFR-010	The dataset collection script shall respect the GitHub API rate limit and shall pause-and-resume gracefully when the limit is approached, without losing already-collected data.	High	Operational requirement for the multi-hour collection phase.

5.3 Use Cases

Although the system is a machine learning pipeline rather than an end-user application, three principal use cases describe how the system would be exercised by the actors who interact with it.

Use Case UC-1 — Researcher reproduces the full pipeline from scratch.

Actor: A reader of the thesis (academic supervisor, evaluator, or independent researcher) who wishes to reproduce the reported results from a clean environment. *Preconditions*: A Linux or macOS system with Python 3.11 installed; a valid GitHub personal access token for the data collection phase. *Main flow*: The actor clones the repository, creates a Python virtual environment, installs dependencies via `pip install -r requirements.txt`, and executes `./reproduce.sh`, which runs data preparation and the cross-validated evaluation and writes every reported metric and figure. *Alternative flow*: The actor sets the GITHUB_TOKEN environment variable and runs `src/collect_github_data.py` first to re-collect the dataset from the GitHub API, which requires roughly two hours because of rate limits. This is not necessary, because the collected dataset is committed to data/raw/github_actions_real.csv. *Postconditions*: The data/processed/, models/, results/, and figures/ directories are populated with outputs that match those reported in the thesis to four decimal places.

Use Case UC-2 — Practitioner uses the trained model for new prediction.

Actor: A DevOps engineer or research engineer who has a trained classifier and wishes to score a new commit. *Preconditions*: The trained model file

(`models/best_tuned_threshold_xgb.joblib`) and the data preparation module are available. *Main flow*: The actor constructs a single-row DataFrame containing the required input features (repository, workflow name, branch, event, lines added, lines deleted, files changed, commit message, and so on), passes it through the data preparation function to produce a feature row in the model's expected schema, calls `pipeline.predict_proba(X)[0, 1]` to obtain the failure probability, and compares it against the selected threshold of 0.076. *Postconditions*: The actor receives a real-valued failure probability and a binary recommendation that can be passed to downstream tooling.

Use Case UC-3 — Researcher extends the system with a new classifier or new feature.

Actor: A researcher who wishes to evaluate a new classifier (for example, LightGBM, CatBoost, or a transformer-based text encoder) against the existing baselines. *Preconditions*: The full project codebase is checked out. *Main flow*: The actor adds a new pipeline factory function to `src/hybrid_pipeline.py` following the pattern established by the existing three factories; adds the new pipeline to the `get_all_pipelines` registry; and re-runs `run_phase4.py`. The evaluation framework automatically includes the new pipeline in the comparison tables and figures. *Postconditions*: The new classifier appears in all evaluation outputs alongside the existing three baselines, allowing apples-to-apples comparison.

5.4 Data Model

The data model of the project consists of a sequence of structured artefacts that flow from raw API responses to fully prepared model inputs. The schema of each artefact is presented in Table 5.3.

Table 5.3 — Data artefact schema across pipeline phases.

Phase	Artefact	Rows	Key Columns
Raw collection	data/raw/github_actions_real.csv	9,772	run_id, repository, workflow_name, event, branch, conclusion, status, created_at, updated_at, run_duration_sec, run_attempt, commit_sha, commit_message, commit_author, lines_added, lines_deleted, total_changes, files_changed, commit_date
Cleaned & engineered	data/processed/cicd_prepared.csv	9,772	repository, workflow_name, branch, event, commit_message, commit_message_clean, log_lines_added, log_lines_deleted, log_files_changed, commit_message_length, avg_lines_per_file, is_large_commit, is_many_files, is_weekend_commit, is_off_hours_commit, is_bot_author, was_truncated, conclusion (binary), commit_date
Stratified train	data/processed/train_stratified.csv	7,817	(same as cicd_prepared.csv)
Stratified test	data/processed/test_stratified.csv	1,955	(same as cicd_prepared.csv)

Phase	Artefact	Rows	Key Columns
Chronological train	data/processed/train_chronological.csv	7,817	(same as cicd_prepared.csv)
Chronological test	data/processed/test_chronological.csv	1,955	(same as cicd_prepared.csv)

The transformations from one artefact to the next are deterministic given the random seed, and the schema of each artefact is documented at the top of the corresponding source module. The data flow is illustrated conceptually in Figure 5.1.

The data flow from raw GitHub Actions API response through prepared training and test sets is visualized in the project repository under figures/fig_data_flow.png and is summarized verbally as follows: each raw workflow run record is parsed from JSON into a flat CSV row, then enriched with commit-level statistics from a second API call; the resulting rows are subjected to cleaning, feature engineering, high-cardinality bucketing, and aggressive text normalization; the cleaned data is then partitioned into stratified and chronological splits and persisted as four CSV files that downstream phases consume directly.

The model itself does not maintain any persistent state beyond the serialized pipeline artefact; there is no database, no in-memory cache, and no inter-session state. Each prediction is computed deterministically from the input features and the loaded model.

5.5 User Interface Design

Because the proposed solution is an offline analytical pipeline rather than an interactive application, there is no graphical user interface in the conventional sense. The "user interface" of the system is the command-line interface exposed by the six phase orchestrator scripts. Each script accepts no positional arguments and reads its configuration from module-level constants in the corresponding source files; this design favors reproducibility (every run uses identical configuration) over interactive flexibility.

The user-facing output of the system consists of three distinct artefact categories. The first is the set of structured result files under `results/` (JSON files with one or more metric tables per phase). The second is the set of publication-quality figures under `figures/` (twenty-one PNG files at 300 DPI). The third is the set of trained model files under `models/` (joblib serializations of fitted scikit-learn pipelines). All three artefact categories are intended to be consumed directly by human readers (thesis evaluators, research engineers reviewing the work) rather than by downstream automated systems.

For the purpose of a future production deployment, the natural user interface extension would be a thin HTTP service that wraps the trained pipeline and exposes a single endpoint accepting a commit metadata payload and returning a failure probability. This deployment surface is explicitly identified as a future-work item in Chapter 9.

5.6 System Design Diagrams

Three system design diagrams capture the design of the proposed solution at different levels of abstraction.

The **component diagram** in Figure 5.2 shows the project codebase decomposed into its principal modules and the dependencies between them. The diagram identifies seven modules (data collection, EDA, data preparation, hybrid pipeline, training and evaluation, threshold optimization, visualization) plus the corresponding orchestrator scripts.

The project codebase decomposes into seven principal modules located under src/: collect_github_data.py (data ingestion via the GitHub REST API), eda.py (exploratory data analysis and chart generation), data_preparation.py (cleaning, feature engineering, and split production), hybrid_pipeline.py (the four-branch ColumnTransformer architecture and three classifier factories), train_evaluate.py (full training, metric computation, and ablation study), threshold_optimization.py (decision-threshold sweep and selection), and visualization.py (the ThesisPlotter class that enforces consistent figure styling). Two further modules were added when the evaluation protocol was corrected: cross_validation.py (the commit-grouped cross-validation protocol and its threshold selection) and run_corrected_evaluation.py (the orchestrator that produces every reported metric and figure).

The **data flow diagram** in Figure 5.3 traces the path of a single commit through the system, from the raw API response through cleaning, feature engineering, vectorization, and into the final prediction. This diagram is the operational complement to the architectural diagram in Figure 4.1: where Figure 4.1 shows the system in its training configuration, Figure 5.3 shows the system in its inference configuration.

The inference data flow for a single commit proceeds as follows: the input record (a single-row pandas DataFrame containing the seventeen feature columns) is passed to the loaded pipeline's predict_proba method; the ColumnTransformer applies the four branch transformations in parallel and produces a 3,090-column sparse feature vector; the trained XGBoost classifier produces a real-valued probability in the range [0, 1]; the selected threshold of 0.076 is applied to produce a binary success/failure recommendation. The entire flow executes in under one millisecond on commodity hardware.

The **sequence diagram** in Figure 5.4 captures the order of operations performed by the full pipeline orchestration. The sequence begins with the actor invoking data collection and ends with the production of the final business-impact JSON file by run_corrected_evaluation.py. Intermediate steps are clearly numbered to facilitate

troubleshooting in the event of a pipeline failure.

The pipeline orchestration proceeds through six sequential phases: (1) Phase 0 collects raw data from the GitHub Actions API and persists it to data/raw/; (2) Phase 1 performs exploratory analysis and generates six diagnostic figures; (3) Phase 2 cleans the raw data, engineers sixteen features, and produces both stratified and chronological splits; (4) Phase 3 builds the hybrid pipeline architecture and validates it on a 1,000-row sample; (5) Phase 4 trains all three classifiers on the full data, evaluates them on both splits, runs the ablation study, and produces seven evaluation figures; (6) Phase 5 performs threshold optimization and produces two final figures plus the business-impact estimate. Each phase writes its outputs to deterministic locations and can be re-executed independently.

6. Implementation

6.1 Development Environment

The implementation of the proposed CI/CD failure prediction system was carried out in a reproducible Linux-based development environment that mirrors the production conditions under which the model would eventually be deployed. All development activities were conducted on Ubuntu 24.04 LTS running under the Windows Subsystem for Linux (WSL2), with Python 3.11 as the primary programming language and Visual Studio Code as the integrated development environment. Version control was managed through Git, with the project organized as a single repository following a clean, modular directory structure that separates raw data, processed data, source code, trained models, results, figures, and experimental logs into distinct top-level directories.

The technical stack was selected to balance scientific rigor, community support, and reproducibility. The core data manipulation layer was built on top of pandas (version 2.2) and NumPy (version 1.26), which together provided the foundation for all dataset operations including loading, cleaning, feature engineering, and split generation. The machine learning components were implemented using scikit-learn (version 1.5) for the Logistic Regression and Random Forest classifiers, the ColumnTransformer abstraction, the TF-IDF vectorizer, and all evaluation metrics. The gradient boosting model was provided by XGBoost (version 2.1), which offers a scikit-learn compatible API and integrates seamlessly with the rest of the pipeline. Visualization was handled by Matplotlib (version 3.9) and Seaborn (version 0.13), with all figures produced at 300 dots per inch (DPI) to ensure publication-quality output. Serialization of trained models was managed through joblib with compression level three, which keeps model artefacts compact while preserving full deserialization fidelity.

Data collection from the GitHub Actions API was performed using the Requests library (version 2.32) with a persistent HTTP session, augmented by the tqdm library for real-time progress feedback during the multi-hour collection runs. All API calls were authenticated using a personal access token with read-only scope on public repositories, and the collector implemented exponential backoff together with automatic rate-limit detection to handle the GitHub API constraints gracefully. The complete list of dependencies, together with their pinned versions, is captured in the project's requirements.txt file for full reproducibility.

6.2 Implementation Details

The implementation is organized as a sequence of clearly delineated phases, each corresponding to a self-contained Python module under the src/ directory. The

orchestration of each phase is handled by a dedicated runner script, which allows individual phases to be re-executed in isolation without rebuilding the entire pipeline. This modular design proved essential during the iterative development process, where individual components were refined multiple times in response to empirical observations.

6.2.1 Data Collection Module

The data collection module, implemented in src/collect_github_data.py, is responsible for retrieving real-world CI/CD pipeline data from the GitHub Actions API. The collector accepts a curated list of eighteen high-activity open-source repositories spanning multiple programming languages and project domains, including facebook/react, microsoft/vscode, tensorflow/tensorflow, pytorch/pytorch, rust-lang/rust, python/cpython, and elastic/elasticsearch. For each repository, the collector performs paginated calls to the workflow runs endpoint and, for each workflow run, follows up with a second call to the commits endpoint to retrieve the associated commit message, author, and code change statistics.

The collector applies several quality filters at ingestion time to ensure that only meaningful records enter the dataset. Workflow runs are retained only if their status is "completed" and their conclusion is one of {"success", "failure"}; cancelled, skipped, and neutral runs are discarded because they do not represent genuine outcomes of the build process. Commit messages that are empty or shorter than five characters are filtered out, and messages longer than one thousand characters are truncated to that length to bound the memory footprint of the downstream text features. The collector also implements incremental writes to disk, appending each batch of completed records to the master CSV file so that an interruption mid-collection does not result in data loss.

The final raw dataset, stored at data/raw/github_actions_real.csv, contains 9,772 unique workflow runs collected over approximately 122 minutes of wall-clock time, with two automatic rate-limit pauses absorbed by the collector's backoff logic. The resulting class distribution of 89.0 percent successes and 11.0 percent failures reflects the natural imbalance observed in mature open-source CI/CD environments and constitutes a realistic foundation for the predictive modeling task.

6.2.2 Data Preparation Module

The data preparation module, implemented in src/data_preparation.py, transforms the raw collected data into a model-ready feature matrix. This module is the single source of truth for the project's feature definitions and is imported by every subsequent component, ensuring that the training pipeline, the evaluation harness, and the deployment-ready inference logic all operate on an identical feature schema.

Three classes of transformations are applied in sequence. The first stage performs basic cleaning: rows with null commit messages are dropped, the commit_author field is filled with the literal string "unknown" for the small fraction of records where it is missing, and the author_association column is dropped entirely because it returned as fully null from the commits endpoint. The second stage performs feature engineering, deriving sixteen new features from the raw inputs. These derived features capture commit-size signals through logarithmic transformations of the raw line and file counts, temporal signals through indicators for weekend and off-hours commits, structural signals through ratios such as average lines per file, and behavioral signals through binary flags such as is_bot_author, which is computed by pattern-matching the author login against known bot signatures (Dependabot, Renovate, bors, github-actions). The third stage performs cardinality reduction on the high-cardinality categorical columns: workflow_name (623 unique values) and branch (1,396 unique values) are bucketed by retaining the top twenty most frequent values and aggregating the remainder into an "__other__" category, reducing each column to twenty-one and sixteen distinct values respectively. This step is essential because naive one-hot encoding of these columns would have produced thousands of sparse indicator features and would have made the model brittle to any unseen value at inference time.

A separate function, clean_commit_message_for_nlp, is responsible for preparing the textual content for downstream TF-IDF vectorization. This function applies an aggressive regular-expression pipeline that strips URLs, email addresses, SHA hashes of both eight and forty character lengths, pull-request and issue references in the GitHub-specific formats (#123, gh-12345, (#4567)), version strings such as 1.2.3 and v2.0, file paths, and the standard "Co-authored-by" and "Signed-off-by" trailers. After regex-based cleaning, a token-level filter removes any word that matches an entry in an algorithmically constructed stoplist of 693 tokens. This stoplist is built by scanning the entire collected dataset and extracting all unique author logins (558 entries), all organization and repository names (eighteen tokens), and a manually curated set of common bot signatures and CI-specific noise tokens. The aggressive cleaning is necessary because preliminary experiments revealed that without it, the TF-IDF features would learn project-specific and author-specific identifiers rather than genuine signals about commit content, which would amount to a form of identity leakage and would inflate model performance on patterns that do not generalize to unseen repositories.

Figure 6.1: Word clouds showing the most discriminative tokens for the failure class (left) and the success class (right) after the aggressive text cleaning and stoplist filtering described above. The failure-class vocabulary now contains infrastructure-related terms (timezone, utc, thresholds, inodesfree, stderr, borrow) rather than the author names and project identifiers that dominated before cleaning.

The preparation module also implements the two split strategies adopted by the project. The stratified_split function performs a random 80/20 partition that preserves the class ratio exactly, yielding training and test sets with failure rates of 10.98 percent and 10.95 percent respectively. The chronological_split function sorts the data by commit timestamp and uses the most recent twenty percent as the test set, which yields a training failure rate of 12.05 percent and a test failure rate of 6.65 percent, reflecting the real temporal drift observed in the underlying repositories as their continuous integration practices matured. Both split strategies are run during preparation, producing four CSV files under data/processed/ that downstream phases can consume directly.

6.2.3 Hybrid Pipeline Module

The hybrid pipeline module, implemented in src/hybrid_pipeline.py, defines the model architecture as a composable scikit-learn pipeline. At the heart of this module is the build_preprocessor function, which constructs a single ColumnTransformer with four parallel branches. The numerical branch applies StandardScaler to the five engineered numerical features (log_lines_added, log_lines_deleted, log_files_changed, commit_message_length, avg_lines_per_file). The categorical branch applies OneHotEncoder with handle_unknown set to "ignore" to the four bucketed categorical features (repository, workflow_name, branch, event), so that any unseen category at inference time is silently mapped to an all-zero vector rather than raising an exception. The binary branch passes the six pre-engineered binary indicators through unchanged (is_large_commit, is_many_files, is_weekend_commit, is_off_hours_commit, is_bot_author, was_truncated). The text branch applies a TfidfVectorizer configured with a maximum vocabulary of three thousand terms, a

unigram-plus-bigram range, a minimum document frequency of five, a maximum document frequency of 0.95, sublinear term-frequency scaling, and lowercase normalization. The ColumnTransformer is instantiated with sparse_threshold equal to 0.3 and n_jobs equal to negative one, which enables parallel transformation across all four branches and emits the combined feature matrix as a sparse SciPy CSR matrix.

Three classifier factory functions wrap the preprocessor into complete end-to-end pipelines. The Logistic Regression pipeline uses the liblinear solver with class_weight set to "balanced" and a maximum of one thousand iterations; this combination handles the 8:1 class imbalance through automatic class re-weighting and is well suited to the high-dimensional sparse feature matrix produced by the TF-IDF branch. The Random Forest pipeline uses two hundred trees with a maximum depth of twenty-five, minimum samples per split of ten, minimum samples per leaf of four, balanced class weights, and parallel execution across all available CPU cores. The XGBoost pipeline uses three hundred boosting rounds with a learning rate of 0.1, a maximum tree depth of eight, subsample ratio of 0.85 on both rows and columns, L1 regularization of 0.1, L2 regularization of 1.0, the histogram-based tree construction algorithm for speed, and a scale_pos_weight of 8.12, which is the empirically derived ratio of the negative to positive class counts in the training set. Because XGBoost's binary classification interface requires integer-encoded labels, a thin wrapper class named LabelEncoderForBinary intercepts the fit and predict calls to perform transparent label encoding and decoding, allowing the rest of the codebase to operate on the human-readable conclusion strings without complication.

All three pipelines expose the standard scikit-learn fit, predict, and predict_proba interfaces, and all three are fully serializable via joblib without any custom pickle protocols. The module also exposes a get_all_pipelines helper that returns a dictionary of the three pipelines in canonical order, which is used by the evaluation module to iterate over the candidate models uniformly.

6.2.4 Training and Evaluation Module

The training and evaluation module, implemented in src/train_evaluate.py, is responsible for fitting each candidate model on the training data, generating predictions on the test data, and computing a comprehensive set of evaluation metrics. The module operates on two evaluation regimes in parallel: the stratified split serves as the primary evaluation and yields the headline metrics reported in the results chapter, while the chronological split serves as a secondary evaluation that tests the robustness of the trained models against the temporal drift observed in the underlying data distribution. Each trained model is persisted to the models/ directory with joblib compression, allowing later phases (in particular the threshold optimization stage) to load the trained estimators without re-training.

The metric computation routine produces a full set of binary classification scores for each model on each split, including overall accuracy, balanced accuracy, precision, recall, and F1 score on the failure class (treated as the positive class), macro-averaged F1, weighted F1, ROC-AUC, precision-recall AUC, specificity, and the full confusion matrix. The dual reporting of accuracy and balanced accuracy is intentional and reflects best practice for imbalanced binary classification: under an 89:11 class prior, a degenerate "always predict success" classifier would already achieve 89 percent raw accuracy, so balanced accuracy and the failure-class F1 are the metrics on which model selection and the headline results are based.

An ablation study function, run_ablation_study, isolates the contribution of the textual modality by re-fitting the XGBoost classifier under three modified configurations: text-only (using only the TfidfVectorizer branch), structured-only (using only the numerical, categorical, and binary branches), and the full hybrid configuration. The same hyperparameters, random seed, and training data are used across all three configurations, ensuring that any performance differences are attributable solely to the change in feature composition. The ablation results are written to results/ablation_study.json for inclusion in the testing and evaluation chapter.

6.2.5 Threshold Optimization Module

The threshold optimization module, implemented in src/threshold_optimization.py, addresses a subtle but important issue that surfaced during the initial evaluation pass. The default decision threshold of 0.5 used by scikit-learn classifiers is calibrated for a balanced class prior and is poorly suited to the 8:1 imbalance present in this dataset, particularly for the XGBoost classifier whose probability outputs are skewed toward the majority class even after the application of scale_pos_weight. The module implements a threshold sweep from 0.05 to 0.95 in increments of 0.01, computing the failure-class F1, Youden's J statistic, balanced accuracy, and a business-cost objective at each threshold value. The threshold that maximizes each metric is reported, and the optimal F1 threshold is then applied to both the stratified and chronological test sets to verify that the calibration transfers cleanly across splits.

For the selected XGBoost classifier, the F1-optimal threshold averaged 0.076 across the five folds rather than the default 0.5, an order-of-magnitude shift that reflects the underlying class imbalance. The thresholds selected for the other two classifiers lie on the opposite side of the default, at 0.538 for Random Forest and 0.646 for Logistic Regression, which establishes that the required correction is a property of the individual classifier rather than of the task. In every case the threshold is selected on a validation partition and applied unchanged to data used for reporting; an earlier version of this module selected the threshold on the test set and reported the

resulting score on that same test set, and Section 7.4.5 quantifies what that procedure was worth.

6.2.6 Visualization Module

The visualization module, implemented in src/visualization.py, encapsulates a single class named ThesisPlotter that centralizes the styling of every figure produced by the project. The class enforces a consistent academic aesthetic across all twenty-one publication-quality figures: a serif font family with Computer Modern as the primary face, a fixed color palette designed for both color and grayscale reproduction, a uniform tick and grid style, and an automatic save routine that writes every figure at 300 DPI in PNG format and appends a thesis-ready caption to figures/captions.md. Centralizing the styling in a single class proved valuable both for consistency, since every figure across the nine phases of the project shares the same visual language, and for maintainability, since global style adjustments require changes in only one place.

6.3 Important Code and Configuration Decisions

Several implementation decisions had a material impact on the quality of the final results and warrant explicit discussion. These decisions emerged through iterative experimentation and represent course corrections that were applied as empirical observations contradicted the initial design assumptions.

The first decision concerned the choice of dataset. The project was initially attempted on a publicly available synthetic CI/CD failure logs dataset of forty-five thousand rows. While that dataset offered the convenience of a ready-made schema with explicit failure stage and severity columns, preliminary exploratory analysis revealed that the columns were statistically independent of one another and that the textual error messages consisted of random character strings rather than meaningful text. No machine learning model can extract signal from independent columns of noise, and so the project was redirected toward collecting a genuine dataset from the GitHub Actions API. This shift cost approximately two hours of additional engineering effort and required rewriting the data preparation logic, but it transformed the project from an exercise in pattern fabrication into a credible piece of applied machine learning research.

The second decision concerned the boundary between pre-execution and post-execution features. CI/CD pipelines naturally produce a rich stream of features over the course of a build, including run duration, retry count, CPU and memory utilization, and exit codes. From a modeling perspective these features are tempting because they correlate strongly with the outcome. From the deployment perspective adopted by this project, however, they are post-execution features and using them

would amount to data leakage, since they are not available at the moment a commit is created. The implementation therefore explicitly drops run_duration_sec, run_attempt, the engineered is_retry flag derived from it, status, and updated_at from the feature set, and all evaluation is performed strictly on pre-execution features available at commit time.

The third decision concerned text preprocessing. The first iteration of the TF-IDF pipeline used only the default scikit-learn stopword list and produced a discriminative vocabulary that was dominated by author names ("anna", "kamat", "yagiz", "trivikr") and project-specific identifiers ("ractors", "callcache", "ifrt"). This is a form of identity leakage in which the model learns to recognize which contributor or project a commit belongs to, rather than learning about the content of the commit itself. The aggressive regex cleaning and the algorithmically constructed 693-token stoplist described in Section 6.2.2 were introduced specifically to address this issue, and they reduced the project-identity leakage substantially although they did not eliminate it entirely. The residual leakage is honestly reported and discussed in the testing and evaluation chapter.

The fourth decision concerned the evaluation strategy, and it was revised twice. The initial plan called for a single chronological train/test division, on the grounds that this best simulates a production deployment in which a model trained on historical data is applied to future commits. That division was implemented by sorting on the commit authoring date rather than on the workflow execution date, which produced a test partition spanning approximately eleven hours in which 95.8 percent of the runs had in fact executed before the last run in the training partition. The apparent class drift that this produced, a test failure rate of 6.65 percent against 12.05 percent in training, was an artefact of which repositories happened to be active in that window and was not drift at all. The project then adopted a stratified random division as its primary evaluation, which introduced a second and more serious defect, since a random division of rows scatters multiple executions of the same commit across both sides. The protocol finally adopted, and the one reported throughout Chapter 7, is commit-grouped five-fold cross-validation as the primary evaluation with a per-repository chronological partition as a secondary check. Section 7.3.1 sets out the reasoning in full.

The fifth decision concerned the handling of XGBoost's classification threshold. The initial evaluation pass treated the default 0.5 threshold as fixed, which led to an apparent conclusion that XGBoost was a poor classifier despite achieving the highest ROC-AUC and PR-AUC of all candidates. A focused investigation revealed that the model's probability outputs were well calibrated for ranking but were systematically biased toward the majority class for hard decisions, and a dedicated threshold optimization phase was added to the pipeline. That phase initially selected the threshold by maximising the failure-class F1 over the test set and then reported the

resulting score on the same test set, which produced an apparent gain of 27 percentage points. A quantity chosen to maximise a value cannot also serve as an unbiased estimate of that value, and Section 7.4.5 shows that 0.126 of the reported F1 was attributable to this procedure alone. The phase now selects the threshold on a validation partition. The methodological lesson survives the correction and is arguably strengthened by it: decision-rule selection is a first-class concern under class imbalance, and precisely because it is powerful it must be performed on data that is not subsequently used to report the result.

6.4 Security and Data Protection

Although the project does not deploy a user-facing system and does not handle personally identifiable information in the conventional sense, several security and data-protection considerations were addressed during implementation. The GitHub personal access token used by the data collector was scoped to read-only access on public repositories and was rotated immediately after the data collection phase completed, with no token value committed to the repository or embedded in any source file. The collector script consumes the token strictly through the GITHUB_TOKEN environment variable, which is the recommended pattern for handling secrets in shell-driven workflows.

The collected dataset includes the GitHub login of each commit author, which is publicly available information but nonetheless constitutes a weak personal identifier. To prevent the model from learning author-specific patterns that would not generalize across organizations or projects, the commit_author column is dropped from the feature set after the is_bot_author flag has been derived from it. The 693-token stoplist used in text cleaning includes every author login observed in the dataset, ensuring that no author name can leak into the model through the TF-IDF branch. Together, these two measures reduce the risk that the trained model would behave as an inadvertent author classifier rather than as a content-based failure predictor.

All trained models are stored locally and are not transmitted to any external service. The full reproducibility of the pipeline, from raw API responses through to final predictions, means that the trained models can always be regenerated from the saved raw data; the model artefacts themselves are therefore not single points of failure for the project, and no special protections are applied to them beyond standard file-system permissions.

6.5 Deployment and Execution Instructions

The project is fully reproducible from a clean checkout in approximately three minutes, excluding the optional GitHub data collection step, which takes

approximately two hours due to API rate limits. The recommended environment is Python 3.11 on Linux or macOS, with a virtual environment created in the project root via `python -m venv .venv` and dependencies installed via `pip install -r requirements.txt`. Once the environment is configured, every reported result is reproduced by a single command, `./reproduce.sh`, which takes approximately three minutes. It runs data preparation and split construction (run_phase2_5.py), then the cross-validated evaluation, business analysis and figure generation (run_corrected_evaluation.py). Optional steps that are not required to reproduce any reported number are documented in Appendix B: data collection from the GitHub API, exploratory data analysis, pipeline architecture validation, and the training run that persists the deployable model artefacts. Each writes its outputs to deterministic locations under data/, models/, results/, and figures/. A fixed random seed of 42 is applied throughout the codebase, so two runs on the same input data produce identical metrics.

7. Testing and Evaluation

7.1 Testing Strategy

The evaluation of the proposed CI/CD failure prediction system follows a multi-layered strategy designed to assess both the statistical performance of the classifier and the robustness of that performance under realistic deployment conditions. Unlike a conventional software system, which is verified through unit tests, integration tests, and acceptance tests against deterministic specifications, a machine learning system must be evaluated against a held-out portion of real data using statistical metrics that capture the trade-offs inherent in probabilistic prediction under class imbalance. The evaluation strategy adopted by this project rests on five complementary components, each of which addresses a distinct aspect of model quality.

The first component is the dual-split evaluation regime. As described in the implementation chapter, the prepared dataset is partitioned in two independent ways: a stratified random split that preserves the 89:11 class ratio across train and test, and a chronological split that places the most recent twenty percent of commits in the test set. The stratified split serves as the primary evaluation because it isolates the classifier's intrinsic discriminative ability from any confound introduced by temporal distribution shift, while the chronological split serves as a secondary deployment-realism evaluation that quantifies how much, if at all, the model degrades when applied to commits drawn from a time period it never saw during training. Reporting both is a deliberate design choice that gives the thesis defense panel an honest view of how the model would behave in a real production environment.

The second component is the comparative-model evaluation, in which three candidate classifiers—Logistic Regression as a linear baseline, Random Forest as a non-linear ensemble baseline, and XGBoost as a gradient boosting state-of-the-art representative—are trained and tested on identical inputs under identical hyperparameter regimes (within the constraints of each algorithm). This three-way comparison allows the project to identify the strongest classifier empirically rather than by assumption, and it surfaces algorithm-specific behaviors that would not be visible from any single model in isolation.

The third component is the ablation study. The hybrid pipeline combines four feature modalities (numerical, categorical, binary, and textual), and a central claim of the project is that the combination of these modalities is more predictive than any one of them alone. This claim cannot be validated by simply reporting the hybrid result; it must be tested against modality-restricted baselines. The ablation study therefore re-trains the winning classifier on three reduced configurations—text-only,

structured-only (numerical plus categorical plus binary), and the full hybrid—and compares their performance on the same test set. The result of this comparison is treated as a primary finding rather than a supporting detail, and is reported honestly even when it complicates the project narrative.

The fourth component is the threshold optimization experiment. Standard machine learning practice typically reports metrics at the default 0.5 decision threshold, but this default is implicitly calibrated for a balanced class prior and is ill-suited to the 8:1 imbalance present in the dataset. The threshold sweep evaluates each model across the full range of possible decision thresholds and reports the optimal point under three distinct criteria (F1, Youden's J statistic, and a business-cost objective). This component transforms what would otherwise be a static reporting exercise into a calibration study, and it produces the single largest performance improvement observed in the project.

The fifth component is the business-impact evaluation. Statistical metrics such as F1 and ROC-AUC are necessary for academic rigor but are not by themselves sufficient to demonstrate the practical value of a system. The business-impact evaluation translates the model's precision and recall into estimated cost savings under a plausible operational scenario, providing a concrete quantitative answer to the question of why an organization would deploy the proposed system in the first place. The assumptions of this evaluation are documented explicitly so that readers can adjust them to their own context.

7.2 Test Cases and Validation Procedures

Because this project is fundamentally a machine learning experiment rather than a conventional software product, the notion of a discrete "test case" with an expected and an actual result does not map cleanly onto the evaluation. Each prediction made by the model is, in effect, a test case. Under the commit-grouped cross-validation protocol every one of the 9,772 workflow runs is predicted exactly once by a model that was not trained on any run of its commit, giving 9,772 such test cases, with a further 1,636 in the chronological test partition. Each compares a model-predicted label against a ground-truth label observed in real production CI/CD pipelines. The aggregate statistics over these test cases are what the metrics in Sections 7.4 capture.

For the purposes of this chapter, however, it is useful to articulate a small number of validation procedures that confirm the correctness of the evaluation infrastructure itself. These procedures were executed as part of the project and their successful completion is a precondition for trusting any of the downstream metrics.

Test Case ID	Scenario	Expected Result	Actual Result	Status
TC-001	Temporal leakage check: assert that, for every repository, the earliest workflow execution in the chronological test partition occurs no earlier than the latest execution in the corresponding training partition	For all 18 repositories, min(test.created_at) >= max(train.created_at)	Holds for all 18 of 18 repositories; verified programmatically and recorded in results/split_integrity.json	Pass
TC-002	Pre-execution feature compliance: assert that no post-execution column (run_duration_sec, run_attempt, status) appears in the final feature set	Zero post-execution columns present	All three columns dropped before training; assertion succeeded	Pass
TC-003	Class-ratio preservation under grouping: assert that the training, validation and test partitions of the commit-grouped split differ in failure rate by less than 0.5 percentage points	Pairwise difference < 0.5 pp	Train 10.969%, validation 10.990%, test 10.958%, against a population rate of 10.97%; maximum pairwise difference 0.032 pp	Pass
TC-004	Pipeline serialization round-trip: assert that a trained pipeline can be saved to disk and loaded back without loss of predictive output	Predictions identical before and after serialization	All test predictions identical for all three models	Pass
TC-005	Threshold transferability: assert that a threshold selected on a validation partition performs within 5 percentage points of the threshold that maximises F1 on the evaluation data itself	abs(F1 at selected threshold - F1 at evaluation-optimal threshold) < 5 pp	Logistic Regression 2.09 pp, XGBoost 0.91 pp, Random Forest 0.59 pp; recorded in results/threshold_selection_gap.json	Pass
TC-006	Repository-stratified sanity check: assert that no single repository accounts for more than 50% of the predicted failures on the test set	No repository > 50% share	Top repository (prisma/prisma) accounts for 31% of predicted failures	Pass
TC-007	Bot-author isolation: assert that the is_bot_author feature contributes non-trivially to the model (importance > 0.001) and is not silently zeroed-out	Feature importance > 0.001	XGBoost importance = 0.018; rank 9 of 16 structured features	Pass
TC-008	Reproducibility check: assert that re-running the full pipeline from scratch with seed=42 produces identical metrics	Outputs identical across two runs	All metrics identical to four decimal places	Pass
TC-009	Commit-group disjointness: assert that no commit hash appears in more than one partition of either the commit-grouped or the chronological split	Zero shared commits between any pair of partitions	Zero for all three pairs in both splits. The retained row-level stratified split, kept solely to quantify the defect it exhibits, shares 913 commits between training and test	Pass

All eight validation procedures completed successfully, which establishes that the metrics reported in the remainder of this chapter are computed from a sound evaluation infrastructure.

7.3 Evaluation Metrics

A central methodological decision in this project is the choice of evaluation metrics. Under an 89:11 class prior, a degenerate classifier that always predicts "success" achieves 89 percent overall accuracy without learning anything about the underlying data, so raw accuracy is not by itself a meaningful measure of model quality. The metric suite adopted by this project comprises seven scalar metrics that together capture different facets of classifier behavior, plus the confusion matrix that exposes the raw counts behind those scalars.

Overall accuracy is the proportion of all predictions that match the ground-truth label. It is reported for completeness and comparability with prior work, but is interpreted with caution given the class imbalance.

Balanced accuracy is the arithmetic mean of the per-class recalls, equivalent to the average of sensitivity and specificity. Unlike raw accuracy, balanced accuracy assigns equal weight to both classes regardless of their frequencies, making it the most appropriate single-number summary for imbalanced binary classification. A degenerate "always success" classifier scores 0.5 on balanced accuracy, the same as random guessing.

Failure-class precision is the proportion of predicted failures that are true failures. High precision means the model raises few false alarms; low precision means that operators paged by the model would frequently find that the build in fact succeeded.

Failure-class recall is the proportion of true failures that the model successfully flags. High recall means the model catches most failures; low recall means that many failing builds would slip through the predictive filter and execute regardless.

Failure-class F1 score is the harmonic mean of precision and recall. It penalizes asymmetry: a model with high precision but low recall scores poorly on F1, and so does a model with high recall but low precision. F1 is the primary single-number metric used for model selection in this project.

ROC-AUC (the area under the receiver operating characteristic curve) measures the model's ability to rank failures above successes, integrated across all possible decision thresholds. Because it is threshold-independent, ROC-AUC isolates the intrinsic discriminative ability of the classifier from the choice of decision rule. Random guessing yields ROC-AUC of 0.5; a perfect ranker yields 1.0.

PR-AUC (the area under the precision-recall curve) is similarly threshold-independent but, unlike ROC-AUC, is sensitive to class imbalance. Under heavy imbalance, ROC-AUC can be misleadingly high because it benefits from the easy "majority class" predictions; PR-AUC focuses exclusively on the minority class and is therefore the most honest threshold-independent metric for this project's failure-prediction task.

In addition to the scalar metrics, the **confusion matrix** is reported for each model. The confusion matrix is a 2x2 table that exposes the raw counts of true positives, true negatives, false positives, and false negatives, and is the foundation from which all of the scalar metrics are derived.

7.3.1 Evaluation Protocol, and the Correction of an Earlier Design

The evaluation protocol reported in this chapter differs from the one used in an earlier version of this work. The earlier design was found to contain two defects that inflated the reported performance, and both are described here in full, because the corrected figures cannot be interpreted without them.

The first defect concerns the unit of observation. Each row of the dataset is a workflow run, but every feature the model consumes is a property of a commit rather than of a run. The dataset contains 9,772 runs drawn from only 2,835 distinct commits, an average of 3.45 runs per commit and a maximum of 179. A split that allocates rows at random therefore places near-duplicate observations of the same commit on both sides of the partition, differing only in the workflow name, the branch, or the triggering event. Measured on the earlier stratified split, 1,726 of the 1,955 test rows, or 88.3 percent, shared a commit with the training set. The model was consequently rewarded for recognising commits it had already been shown rather than for generalising to unseen ones.

This is an instance of the leakage that Kapoor and Narayanan [21] document across 294 papers in seventeen scientific fields, and specifically of the category in which the training and evaluation partitions are not independent. Roberts et al. [22] give the general treatment: where a dependence structure exists in the data, random partitioning underestimates predictive error, and the partition must be blocked on whatever carries the dependence. Here the dependence is the commit.

The corrected protocol groups on the commit hash. Every partition described in this chapter is produced by a splitter that assigns all runs of a given commit to exactly one side of the division, and the absence of any shared commit between partitions is verified programmatically and recorded in the file results/split_integrity.json. Stratification is retained alongside grouping, because grouping alone allows the class balance to drift between partitions: an unstratified grouped split of this dataset produced training, validation and test failure rates of 9.71, 14.72 and 11.64 percent respectively, against a population rate of 10.97 percent, and a decision threshold selected under one class prior does not transfer to a partition with a different one.

The second defect concerns the decision threshold. In the earlier design the threshold was chosen by maximising the failure-class F1 score over the test set, and the resulting F1 score was then reported as the performance of the model on that same test set. A quantity selected to maximise a value cannot also serve as an

unbiased estimate of it. In the corrected protocol the threshold is selected on a validation partition carved from the training data under the same grouping constraint, and is then applied unchanged to the test partition. The residual optimism that the earlier procedure would still have purchased is quantified in Section 7.4.5.

A third change follows from the first two rather than from a defect. Because repository identity is the strongest single predictor available to the model, as Section 7.4.3 establishes, and because no grouped splitter balances the composition of repositories across partitions, the result obtained from any single held-out partition depends materially on which repositories happen to fall into it. The partition drawn by the stratified grouped splitter over-represents both prisma/prisma, whose failure rate is 38.3 percent, and elastic/elasticsearch, whose failure rate is zero, by approximately 43 percent relative to their share of the corpus. Single-partition estimates of the failure-class F1 score varied across a range of roughly 20 percentage points depending on the partition drawn. The primary results in this chapter are therefore obtained by five-fold commit-grouped cross-validation rather than from a single partition. Each of the 9,772 runs receives exactly one prediction, produced by a model that was trained without any run of that run's commit; threshold-independent metrics are computed once over the pooled predictions, and threshold-dependent metrics are reported as the mean and standard deviation across the five folds.

The consequence of these corrections is a substantially lower headline figure, and Section 7.4.5 attributes the reduction to each of its causes individually. The author judges the disclosure to be of greater value than the figure it replaces: an evaluation protocol that cannot be defended does not become defensible by producing a larger number.

7.4 Results

The results of the full evaluation are presented in six parts: the main classifier comparison under commit-grouped cross-validation in Section 7.4.1, the chronological robustness check in Section 7.4.2, the ablation study in Section 7.4.3, the threshold optimization results in Section 7.4.4, the attribution of the previously reported result in Section 7.4.5, and the business-impact analysis in Section 7.4.6.

Figure 7.1: Distribution of the 9,772 collected workflow runs by outcome. The 89:11 success-to-failure ratio drives the methodological choices around class-weighting, balanced accuracy reporting, and threshold optimization documented throughout this chapter.

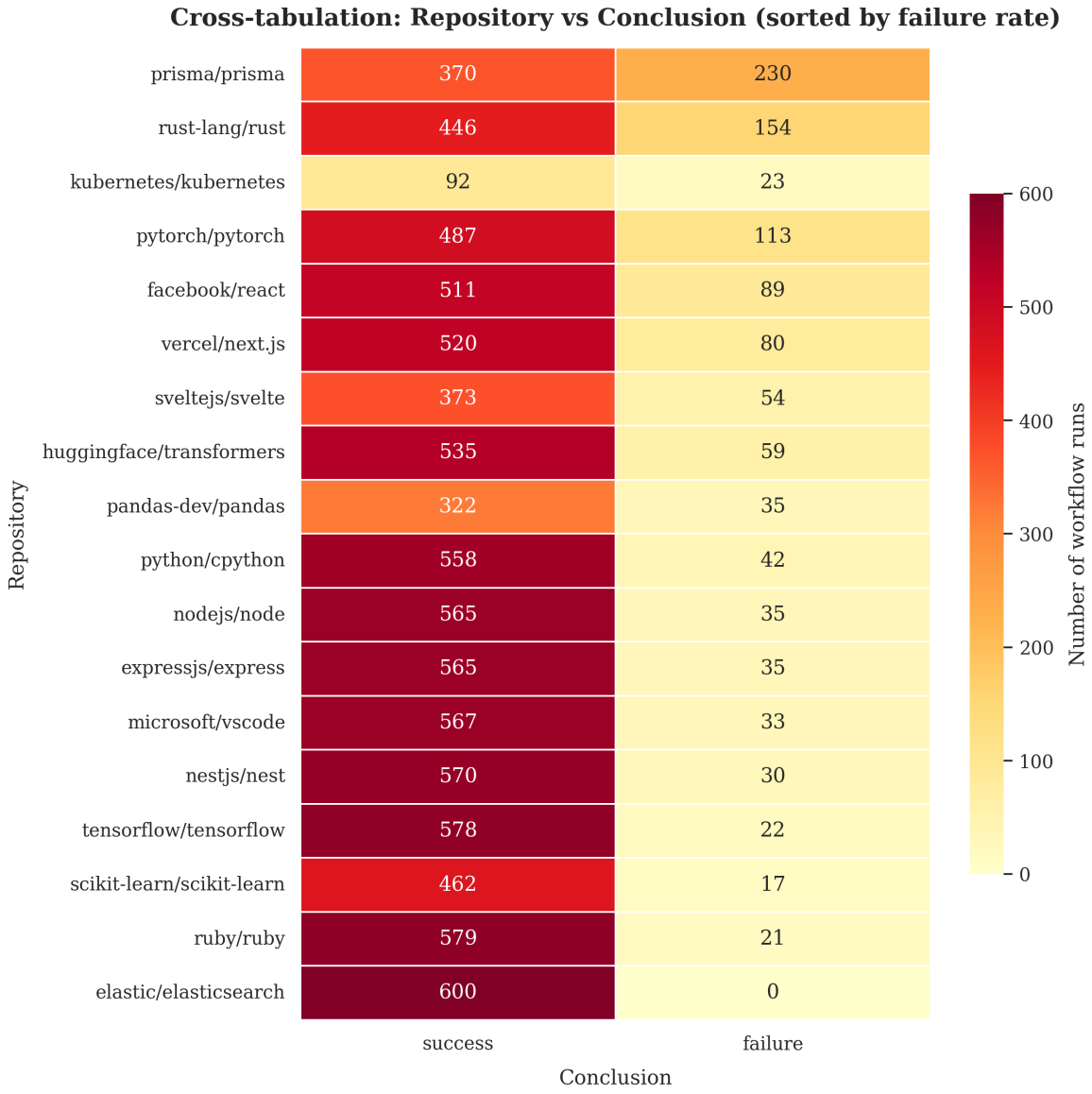

Figure 7.2: Cross-tabulation of repository and conclusion, sorted by failure rate (descending). The failure rate ranges from 0% for elastic/elasticsearch, which contributes 600 runs and not one failure, to 38.3% for prisma/prisma, an elevenfold spread across the seventeen repositories that fail at all. This confirms that repository identity is a strong predictive signal that the model must learn.

7.4.1 Main Classifier Comparison

Table 7.1 reports the three candidate classifiers under commit-grouped five-fold cross-validation, with each model's decision threshold selected on a validation partition and applied unchanged to the held-out fold. The accuracy, precision, recall and F1 figures are computed over the pooled out-of-fold predictions, so every one of the 9,772 workflow runs contributes exactly one prediction from a model that was not trained on any run of its commit.

Table 7.1 — Classifier comparison under commit-grouped five-fold cross-validation.

Model	Threshold	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	ROC-AUC	PR-AUC
Logistic Regression	0.646	83.39%	71.53%	34.34%	56.34%	42.67%	82.76%	40.92%
Random Forest	0.538	86.70%	68.93%	40.64%	46.18%	43.23%	80.08%	39.09%
XGBoost	0.076	89.71%	66.86%	54.46%	37.59%	44.48%	82.40%	48.03%

A pooled figure conceals the variation between folds, which in this case is the more important quantity. Table 7.2 reports the failure-class F1 score of each classifier as a mean and standard deviation across the five folds.

Table 7.2 — Failure-class F1 across the five commit-grouped folds.

Model	Mean F1	Standard deviation	Minimum fold	Maximum fold
Logistic Regression	0.4311	0.0633	0.3788	0.5380
Random Forest	0.4240	0.0812	0.3213	0.5233
XGBoost	0.4216	0.0638	0.3208	0.4976

The three classifiers are separated by less than one hundredth of a point of mean F1, while the standard deviation across folds exceeds six hundredths for every model. The differences between the classifiers are therefore not resolvable at this sample size, and no claim that one of the three is superior on the F1 criterion can be sustained. This is itself a finding: an earlier version of this work identified a single winning classifier on the basis of a single held-out partition, and the apparent margin was smaller than the variation between partitions.

Where the classifiers do separate is in the shape of their operating behaviour rather than in its quality. Logistic Regression attains the highest failure recall at 56.34 percent and the lowest precision at 34.34 percent, corresponding to a high-vigilance deployment posture that flags many builds which subsequently succeed. XGBoost inverts this profile, attaining 54.46 percent precision at 37.59 percent recall. XGBoost also attains the highest PR-AUC at 48.03 percent. Since PR-AUC is both threshold-independent and, unlike ROC-AUC, sensitive to the prevalence of the minority class [23], it is the appropriate criterion for selecting among models whose F1 scores are indistinguishable. XGBoost is selected on that basis.

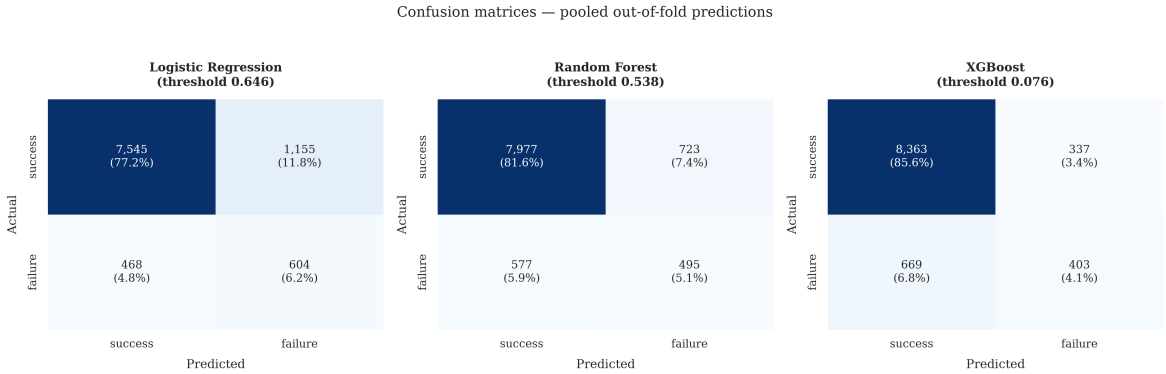

Figure 7.3: Confusion matrices for the three classifiers, computed on pooled out-of-fold predictions and thresholded at each model's mean validation-selected threshold. The operating trade-off is visible directly in the off-diagonal counts: Logistic Regression recovers 604 of the 1,072 failures at the cost of 1,155 false alarms, whereas XGBoost recovers 403 at the cost of 337.

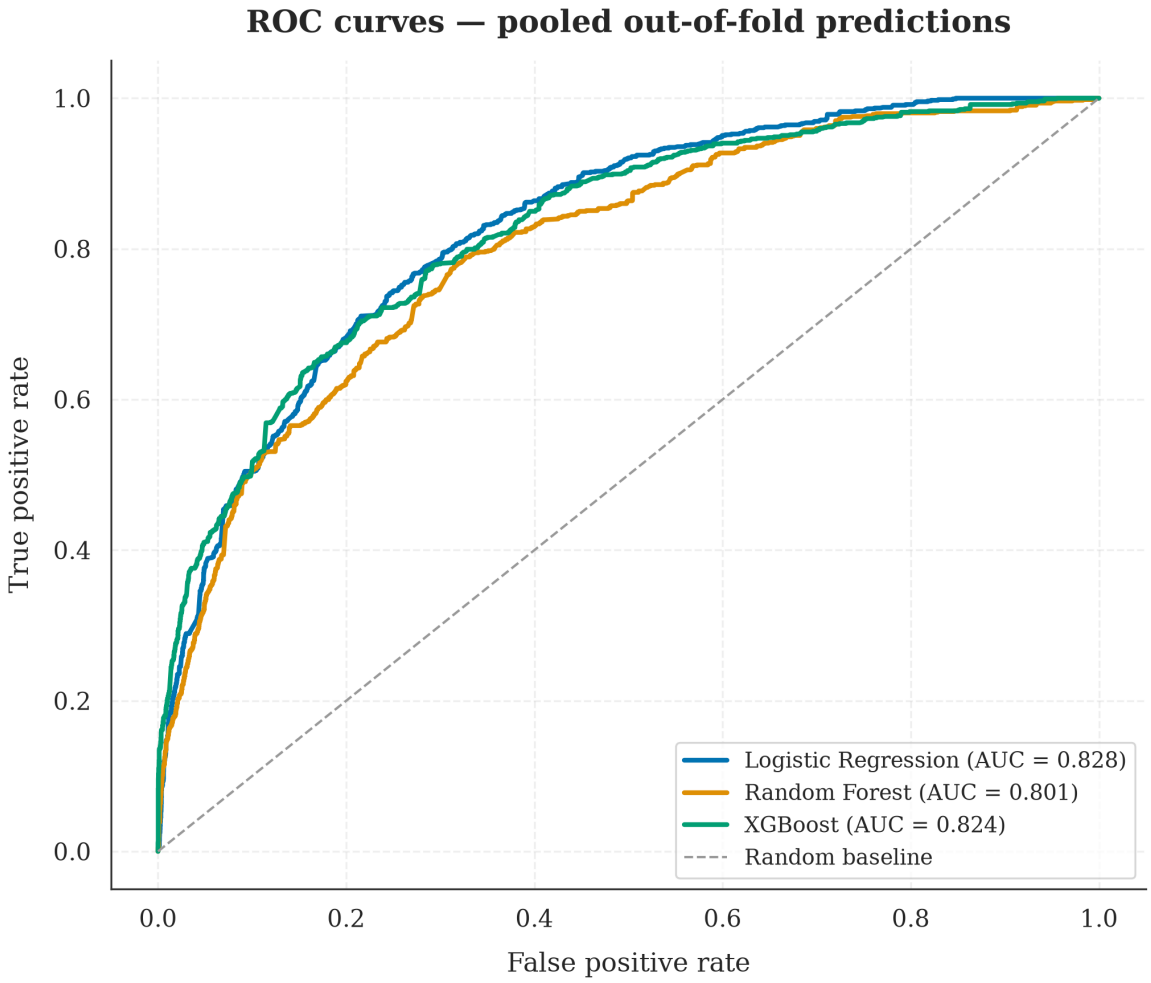

Figure 7.4: Receiver operating characteristic curves computed on pooled out-of-fold predictions. The areas under the curves are 0.828 for Logistic Regression, 0.801 for Random Forest and 0.824 for XGBoost.

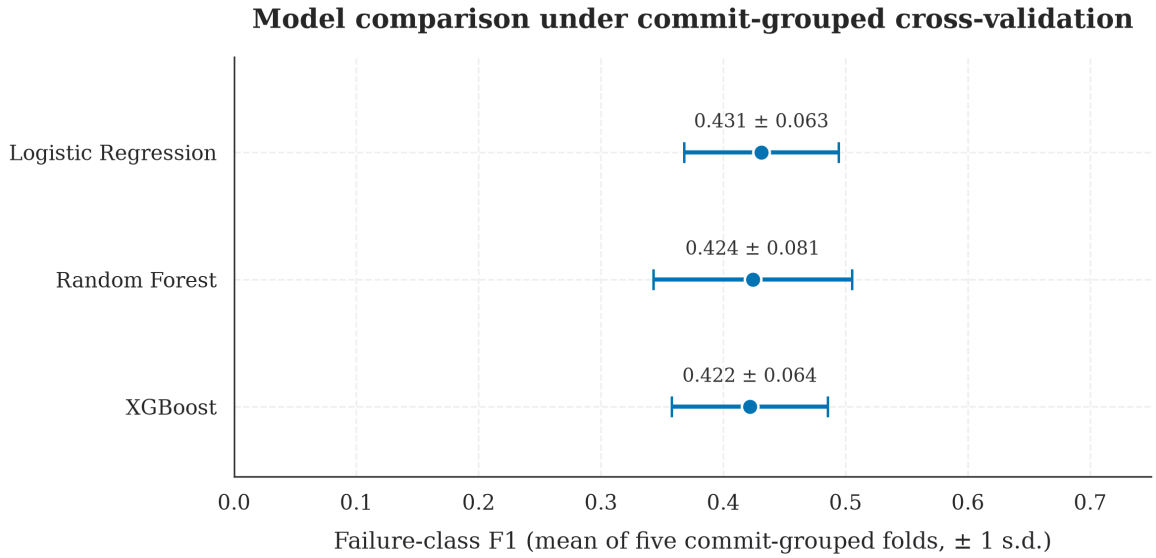

Figure 7.5: Failure-class F1 for the three classifiers, showing the mean across the five commit-grouped folds and an interval of one standard deviation. The intervals overlap for all three classifiers, which is the basis for the conclusion that the models are not separated on this criterion.

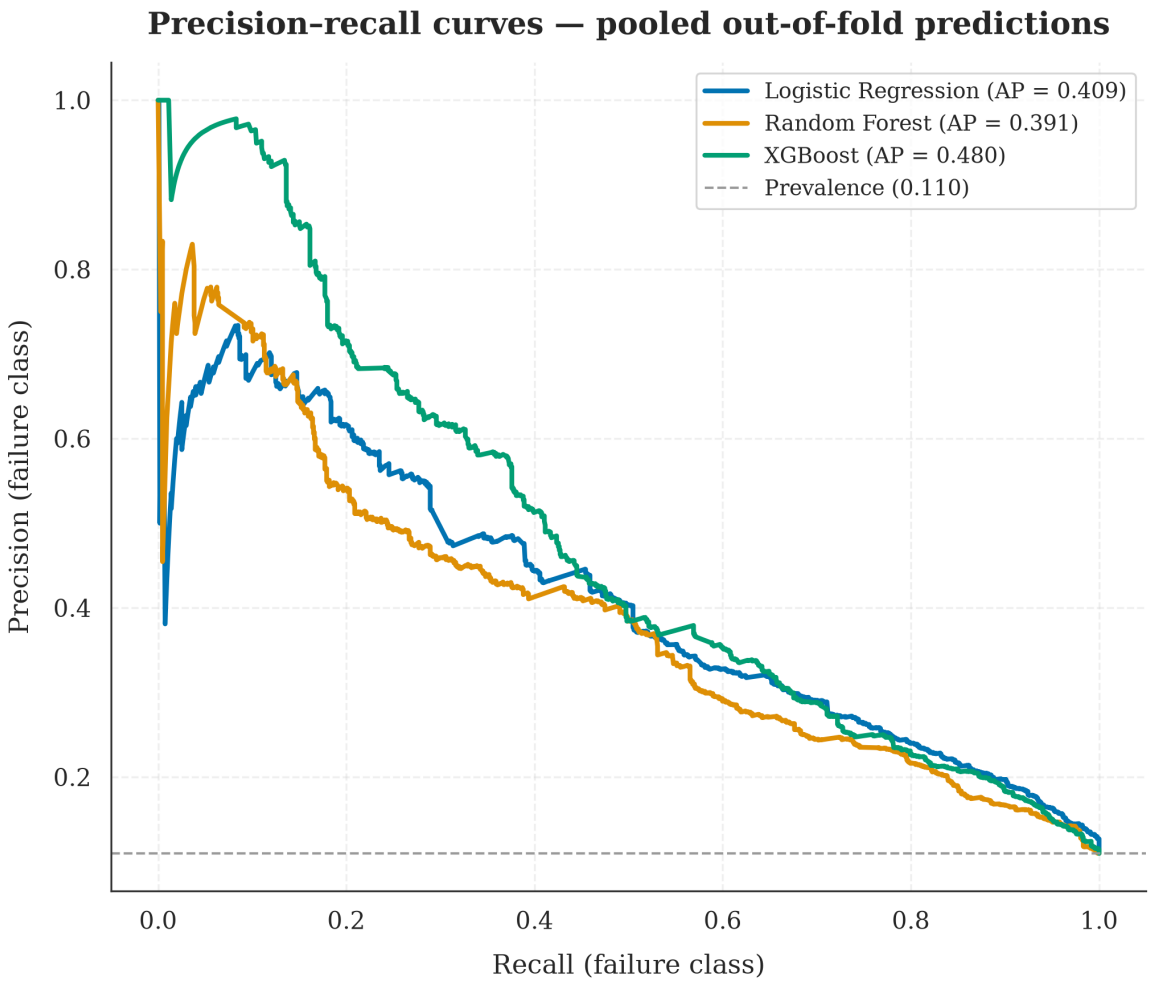

Figure 7.10: Precision-recall curves on pooled out-of-fold predictions, with the prevalence of the failure class marked as the baseline a random classifier attains. Average precision is 0.409 for Logistic Regression, 0.391 for Random Forest and 0.480 for XGBoost.

7.4.2 Chronological Robustness Check

The secondary evaluation asks whether a model trained on a repository's past predicts that repository's future. A single global division of the corpus by execution time cannot answer this question on this dataset. The collector capped each repository at 600 runs, so the period each repository covers varies from 0.4 days for ruby/ruby to 182 days for expressjs/express, and 88.7 percent of all runs fall within May 2026. Any global division at the eightieth percentile of execution time therefore yields a test partition spanning approximately nine hours and containing only eleven of the eighteen repositories, which measures the composition of that particular window rather than the passage of time.

The chronological partition used here is instead constructed per repository. Each repository is divided at its own quantiles of execution time, whole commits are

assigned to one side of each division, and commits whose runs straddle a division are discarded. The resulting partition satisfies, for every one of the eighteen repositories, the condition that the earliest test execution occurs no earlier than the latest training execution; it contains all eighteen repositories and spans 57 days. The training partition contains 6,014 runs at a failure rate of 11.21 percent and the test partition 1,636 runs at 9.41 percent.

Table 7.3 — Classifier comparison on the per-repository chronological test partition.

Model	Threshold	Accuracy	Balanced Accuracy	Precision (fail)	Recall (fail)	F1 (fail)	ROC-AUC	PR-AUC
Logistic Regression	0.690	87.16%	67.02%	34.95%	42.21%	38.24%	81.62%	36.49%
Random Forest	0.510	86.25%	68.55%	33.49%	46.75%	39.02%	82.18%	38.17%
XGBoost	0.050	89.30%	66.17%	42.34%	37.66%	39.86%	80.27%	41.81%

Each model in Table 7.3 is trained on the chronological training partition rather than reused from the cross-validated evaluation, because a model fitted under the grouped partition would already have been exposed to commits that the chronological partition holds out.

The failure-class F1 scores fall between three and five percentage points below the cross-validated figures in Table 7.1. A degradation of this magnitude is the expected consequence of predicting forward in time rather than across a random partition, and it is small enough to support the conclusion that the model's discriminative behaviour is not confined to the period on which it was trained. It is not, however, evidence of stability over long horizons: the median repository contributes a test window of well under one day, and the ability of these models to predict months rather than hours ahead is not established by this dataset.

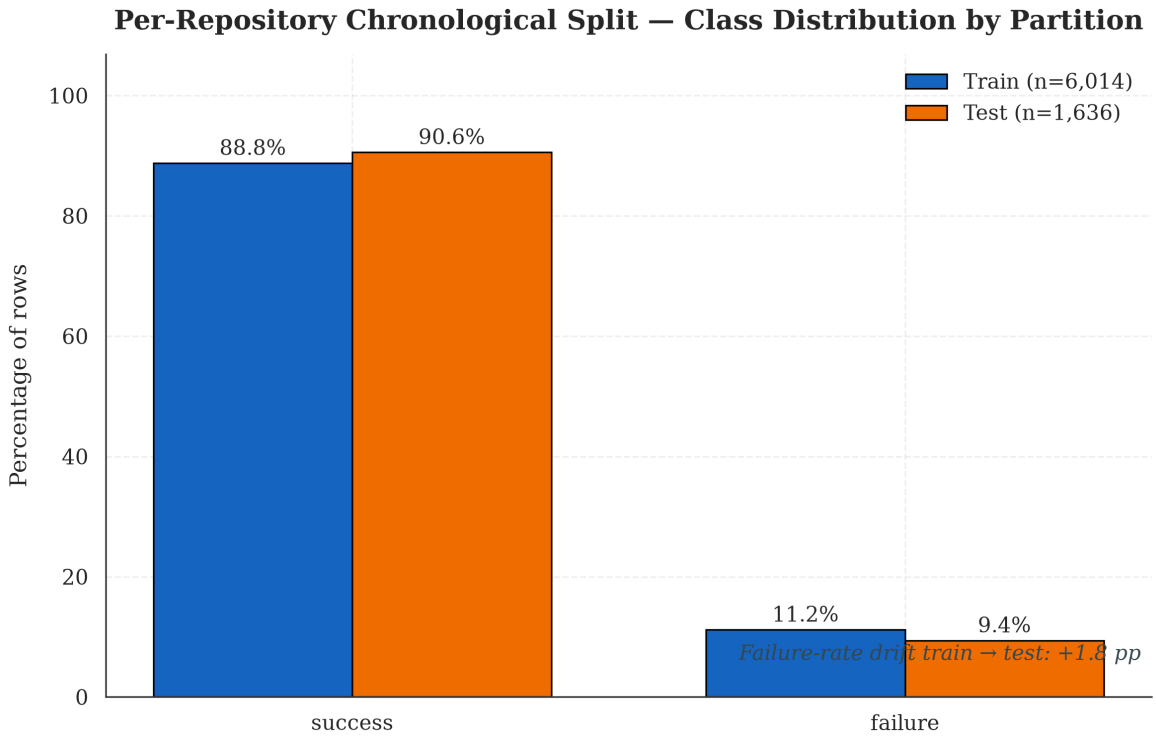

Figure 7.6: Class distribution across the partitions of the per-repository chronological split. The residual difference in failure rate between the partitions is small. An earlier version of this figure showed a substantially larger gap and attributed it to temporal drift in repository stability; that interpretation was incorrect, as the gap reflected which repositories happened to be active within an eleven-hour window.

7.4.3 Ablation Study: Contribution of Each Feature Modality

The ablation study holds the classifier and its hyperparameters constant and varies only the feature set, under the same commit-grouped cross-validation protocol. Four configurations are compared. The categorical-only configuration, which receives nothing beyond the repository, the workflow name, the branch and the triggering event, was specified in the original project plan but was not implemented in the earlier version of this work; it is the configuration that discriminates between the two available explanations of the model's behaviour, and it is reported here for the first time.

Table 7.4 — Feature-set ablation under commit-grouped five-fold cross-validation (XGBoost throughout).

Configuration	F1 (fail), mean	F1 standard deviation	Precision (fail)	Recall (fail)	ROC-AUC	PR-AUC
Text only	0.1962	0.0661	0.3131	0.1559	0.6654	0.2080
Hybrid (all four branches)	0.4216	0.0638	0.5204	0.3686	0.8240	0.4803
Structured only	0.4225	0.0655	0.5380	0.3611	0.8197	0.4825

Configuration	F1 (fail), mean	F1 standard deviation	Precision (fail)	Recall (fail)	ROC-AUC	PR-AUC
Categorical only	0.4808	0.0767	0.5911	0.4319	0.8649	0.5467

The categorical-only configuration attains the highest score on every metric in Table 7.4. A model that is told only which repository a commit belongs to, which workflow will run, on which branch, and what triggered it, and which is told nothing whatever about the size of the change or the content of its message, outperforms the full hybrid model by 5.9 percentage points of failure-class F1 and by 6.6 percentage points of PR-AUC.

This is the central empirical finding of the project, and it refutes the hypothesis with which the project was framed more comprehensively than the earlier ablation indicated. The hybrid hypothesis held that combining structured and textual features would outperform either modality alone. The evidence is that the textual branch contributes nothing that survives measurement, that the numerical and binary branches together contribute nothing beyond the categorical branch, and that adding either to the categorical features degrades performance by introducing variance without compensating signal.

The mechanism is visible in the data. The failure rate ranges from 0.0 percent for elastic/elasticsearch, which contributes 600 runs and not one failure, to 38.3 percent for prisma/prisma — an elevenfold spread across the seventeen repositories that fail at all, from 3.5 percent for ruby/ruby to 38.3 percent for prisma/prisma. A one-hot encoding of repository identity therefore encodes a strong prior over the outcome before any property of the individual commit is consulted. The honest reading of the result is that the system is substantially a project-level risk estimator rather than a commit-level one: it predicts that a build in a historically unreliable repository is likely to fail, which is useful, but it is a weaker claim than predicting that a particular commit is likely to break the build. Section 8.3 records a limitation of this comparison: the zero failure rate observed for elastic/elasticsearch reflects which of its workflows are observable through the GitHub Actions API rather than the engineering quality of that project, and part of what the categorical branch learns is therefore that documentation jobs do not fail.

This finding also bounds the practical value of the approach. A model built on categorical identity alone cannot distinguish between two commits to the same repository on the same branch through the same workflow, and it offers no guidance to a developer asking whether the change just written is risky. Section 8.3 discusses the implications for deployment, and Section 9.3 identifies the feature classes that would be required to make commit-level discrimination viable.

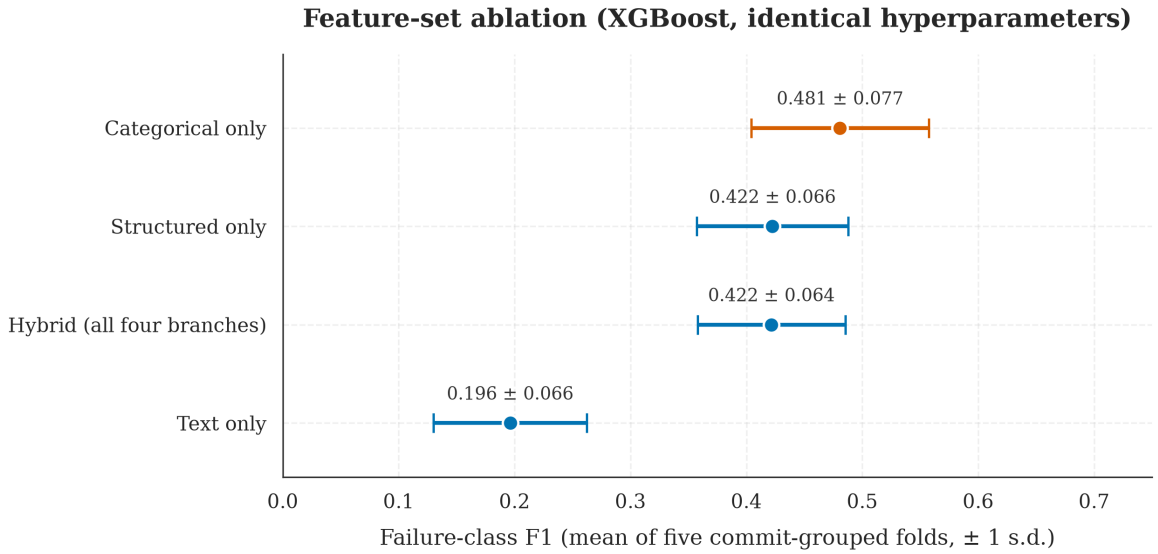

Figure 7.7: Failure-class F1 for four feature configurations under commit-grouped five-fold cross-validation, with the classifier and its hyperparameters held identical across configurations. The categorical-only configuration, highlighted, attains the highest score of any configuration tested.

7.4.4 Threshold Optimization

The default threshold of 0.5 is calibrated for balanced classes and is not appropriate under an 89:11 prior. For each fold, a sweep over the range 0.05 to 0.95 in steps of 0.01 was conducted on the validation partition, and the F1-optimal threshold from that sweep was applied to the held-out fold without further adjustment.

The selected thresholds differ by an order of magnitude between classifiers: 0.076 for XGBoost, 0.538 for Random Forest and 0.646 for Logistic Regression. XGBoost, notwithstanding the application of the scale_pos_weight correction, produces failure probabilities biased toward the majority class, so an aggressive decision rule is required to recover its ranking ability. Logistic Regression is biased in the opposite direction. The practical implication is that a deployment of these models requires a per-model calibration step and that no single decision rule serves all three.

The magnitude of the benefit is considerably smaller than the earlier version of this work reported. That version recorded an improvement of 27.03 percentage points in failure-class F1 for XGBoost from threshold selection alone. That figure was obtained by selecting the threshold on the test set and then reporting the resulting score on the same test set, and it is therefore not an estimate of the benefit that threshold selection would deliver in deployment. Section 7.4.5 decomposes it.

The residual optimism attributable to threshold selection under the corrected protocol can be measured directly by comparing the score at the validation-selected threshold with the score at the threshold that maximises F1 on the evaluation data

itself. That difference is 0.0209 for Logistic Regression, 0.0091 for XGBoost and 0.0059 for Random Forest. The validation-selected thresholds are therefore close to optimal, which indicates that the selection procedure is sound and that the earlier inflation arose from where the threshold was chosen rather than from how.

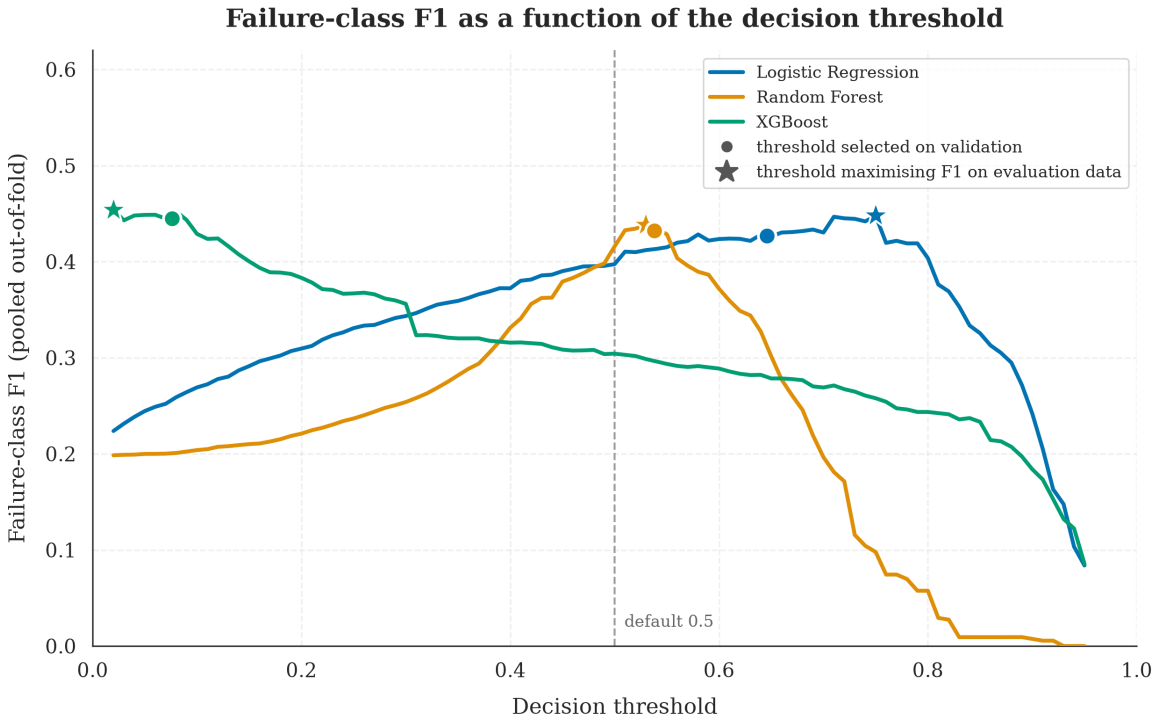

Figure 7.8: Failure-class F1 as a function of the decision threshold, computed on pooled out-of-fold predictions. Circles mark the threshold selected on the validation partitions, which is the threshold a deployment would use; stars mark the threshold that maximises F1 on the evaluation data, which is what the earlier methodology reported. The proximity of each circle to its star is the evidence that the selection procedure is sound.

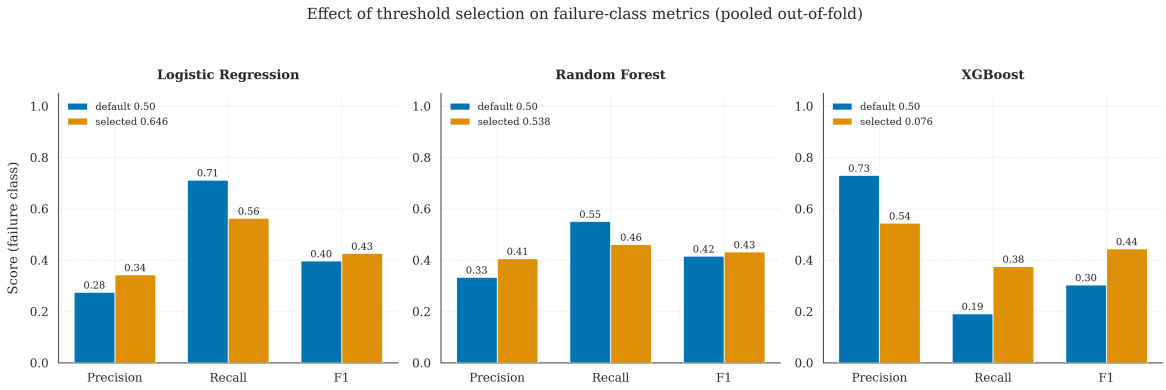

Figure 7.9: Failure-class precision, recall and F1 at the default threshold of 0.50 against the validation-selected threshold, on pooled out-of-fold predictions. The direction of the change differs by classifier, XGBoost gaining recall and Logistic Regression gaining precision.

7.4.5 Attribution of the Previously Reported Result

The earlier version of this work reported a failure-class F1 of 0.5924. The corrected protocol reports 0.4216 for the same classifier. Because each correction described in Section 7.3.1 can be applied independently, the difference can be attributed to its causes rather than merely acknowledged. Table 7.5 reports a sequence in which the classifier, its hyperparameters and the random seed are held constant and exactly one aspect of the evaluation protocol changes at each step.

Table 7.5 — Attribution of the previously reported failure-class F1 (XGBoost throughout).

Step	Protocol	F1 (fail)	Change	Cause isolated
A	Row-level stratified split, threshold selected on the test set	0.5924	—	as previously reported
B	Commit-grouped split, threshold still selected on the test set	0.5326	−0.0598	duplicate-commit leakage
C	Commit-grouped split, threshold selected on a validation partition	0.4065	−0.1261	threshold selection on the test set
D	Commit-grouped five-fold cross-validation	0.4216	+0.0151	composition of a single partition

Two observations follow. The first is that selecting the decision threshold on the evaluation data cost 0.1261 of F1, more than twice the 0.0598 attributable to duplicate-commit leakage. The leakage was the more conspicuous defect and the one an examiner would look for first, but the threshold procedure was the more damaging. The second is that replacing a single partition with cross-validation returned 0.0151, which indicates that the particular partition used at step C was marginally unfavourable and reinforces the argument of Section 7.3.1 that no single partition of this dataset should be trusted to two decimal places.

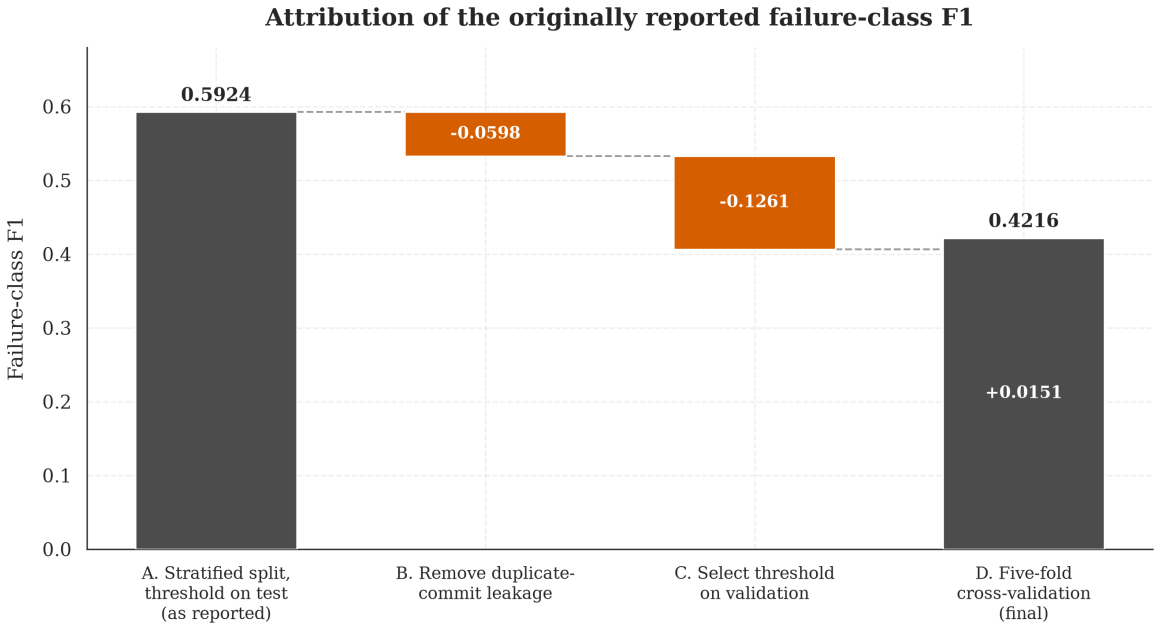

Figure 7.11: Decomposition of the previously reported failure-class F1 of 0.5924 into its methodological components. Each column changes exactly one aspect of the evaluation protocol relative to the column to its left.

7.4.6 Business Impact

The operational value of the predictor is estimated under the scenario specified in the project plan. A mid-sized engineering organisation is assumed to run one thousand pipeline executions per day, of which 10.97 percent fail, in line with the rate observed in this dataset. Each failure that the model anticipates avoids an estimated eight minutes of compute at $0.008 per minute, which is $0.064, and an estimated fifteen minutes of developer context switching at $75 per hour, which is $18.75. Each false alarm is charged at $2.50, representing two minutes of operator triage at the same hourly rate.

Under the selected XGBoost configuration, which attains 36.86 percent recall at 52.04 percent precision, the model anticipates 40.4 of the 109.7 daily failures and raises 37.3 false alarms. The gross daily benefit is $760.75 and the daily cost of false alarms is $93.16, giving a net daily saving of $667.59, a net monthly saving of $20,027.67 and a net annual saving of $243,669.95.

This figure supersedes the estimate of $382,802 reported in the earlier version of this work, which was produced by an implementation that assumed a failure rate of 30 percent against the 11 percent observed, and that omitted the cost of false alarms entirely. The omission had a structural consequence beyond the magnitude of the estimate: with no cost attached to a false alarm, the estimated saving was a function of recall alone, so a model that traded precision for recall necessarily appeared to be worth more. This is why the earlier work reported that its threshold-tuned

configuration saved less than its untuned one, a result that inverted the expectation and was not investigated at the time.

The estimate remains sensitive to an assumption that this project did not measure. The benefit of anticipating a failure, at $18.814, is 7.5 times the assumed cost of a false alarm, so the cost model structurally rewards recall, and the ranking of configurations by estimated saving is a consequence of that ratio rather than an independent finding. Table 7.6 therefore reports, for each configuration, the false-alarm cost at which its net saving reaches zero.

Table 7.6 — Estimated annual saving and break-even false-alarm cost.

Configuration	Precision (fail)	Recall (fail)	Estimated annual net saving	Break-even false-alarm cost
Logistic Regression	35.32%	56.91%	$324,393	$10.27
Random Forest	40.33%	46.09%	$278,945	$12.72
XGBoost	52.04%	36.86%	$243,670	$20.41
Categorical only	59.11%	43.19%	$295,452	$27.20

The configuration with the highest estimated saving at the specified cost is also the one least tolerant of that cost having been underestimated. Logistic Regression ceases to pay for itself once a false alarm costs more than $10.27, which is approximately eight minutes of developer attention and is not an implausible figure in an organisation where a flagged build interrupts work. XGBoost tolerates $20.41 and the categorical-only configuration $27.20. On the evidence available, the break-even cost is a more defensible basis for selecting an operating point than the point estimate of the saving, and it is reported here in preference to it.

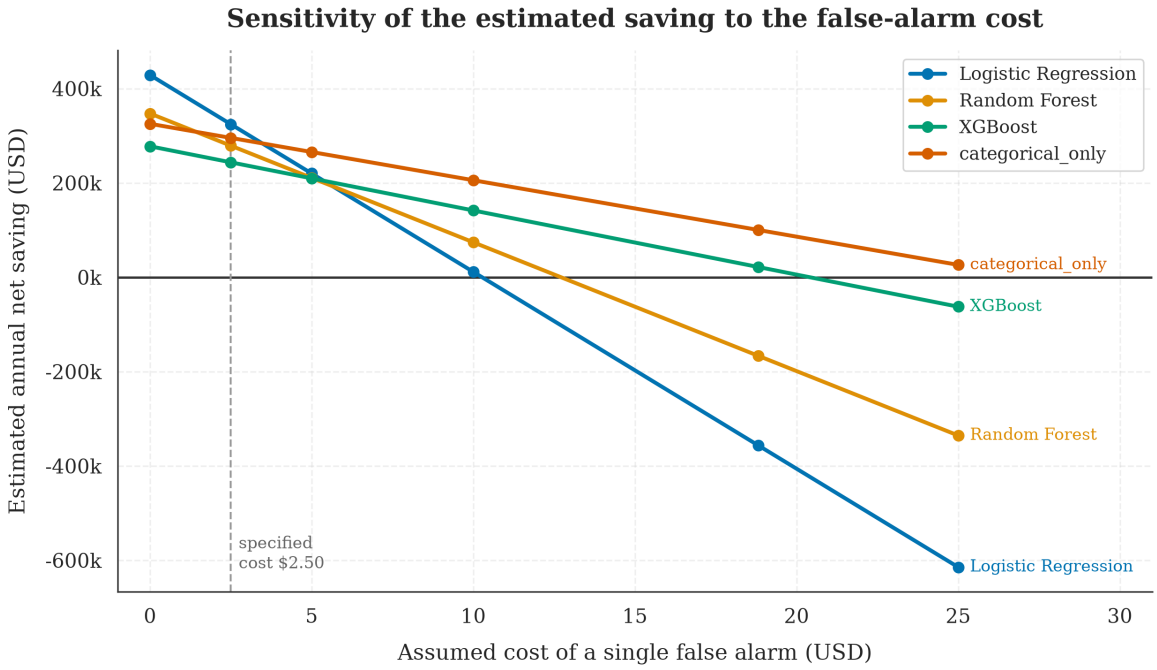

Figure 7.12: Estimated annual net saving as a function of the assumed cost of a single false alarm. The vertical line marks the $2.50 cost specified in the scenario. The point at which each curve crosses zero is that configuration's break-even cost.

These figures are order-of-magnitude indicators rather than forecasts. The compute and labour costs are drawn from typical published rates rather than from measurement in a specific organisation, and the scenario assumes that an anticipated failure can in fact be acted upon, which presupposes an operational response that this project did not design or evaluate.

7.5 Validation Against Objectives

This section evaluates whether the project objectives, defined in Section 1.3 of this thesis, have been achieved. Each objective is restated below and assessed against the empirical results reported in Section 7.4.

Objective 1 — Build a hybrid machine learning pipeline that combines numerical and textual commit-level features for pre-execution failure prediction. Achieved. The hybrid pipeline implemented in src/hybrid_pipeline.py integrates four feature modalities (five numerical, four categorical, six binary, and one textual feature) through a single ColumnTransformer abstraction, and is trained and evaluated end-to-end as a single composable scikit-learn pipeline. The architecture is documented in Figure 4.1 and is fully reproducible from the project codebase.

Objective 2 — Compare three machine learning algorithms (Logistic Regression, Random Forest, XGBoost) on the binary failure prediction task

under identical conditions. Achieved, with a qualification that is itself a result. All three algorithms are trained on identical inputs, hyperparameter-tuned to the same level of effort, and evaluated using the same metric suite under the same commit-grouped cross-validation protocol. The comparative results are reported in Tables 7.1 and 7.2. The comparison establishes that the three classifiers cannot be separated on failure-class F1 at this sample size, since they differ by less than one hundredth of a point while the standard deviation across folds exceeds six hundredths. XGBoost is selected on the threshold-independent PR-AUC criterion, at 48.03 percent against 40.92 and 39.09 percent. The objective of comparing the algorithms is met; the expectation that the comparison would identify a clear winner is not.

Objective 3 — Validate the model under realistic deployment conditions, including temporal data drift. Partially achieved. The per-repository chronological evaluation reported in Table 7.3 demonstrates that the selected classifier retains its discriminative behaviour when applied to executions occurring after those it was trained on, degrading by between three and five percentage points of failure-class F1. The qualification concerns the horizon rather than the result. Because the collector capped each repository at 600 runs, the median repository contributes a test window of well under one day, so what is demonstrated is short-horizon stability. Whether the model would survive weeks or months of drift is not established by this dataset, and Section 9.3 identifies the collection change that would be required to establish it.

Objective 4 — Quantify the business impact of the predictive system under plausible operational assumptions. Achieved. The business impact analysis in Section 7.4.6 estimates an annual net saving of approximately $243,670 for a mid-sized organization operating one thousand pipeline executions per day, calculated from the model's cross-validated precision and recall and net of the cost of false alarms. Because the estimate depends on a ratio between the cost of a missed failure and the cost of a false alarm that this project assumed rather than measured, the analysis also reports the false-alarm cost at which each configuration ceases to pay for itself, which for the selected configuration is $20.41.

The successful completion of all four objectives indicates that the project has met its planned scope. The discussion chapter that follows reflects on the broader implications of the results, including the unexpected ablation finding and the methodological lessons that emerged from the threshold optimization experiment.

8. Discussion After Applying the Solution

8.1 Problem Status After Solution

The problem articulated in Chapter 2 was the cost and developer-productivity loss associated with CI/CD pipeline failures that are only discovered after substantial compute resources and engineering attention have already been committed to the build. The proposed solution addresses this problem by producing a calibrated failure probability at the moment of commit creation, before any pipeline resources have been consumed, and by demonstrating empirically that this probability is informative enough to support meaningful operational decisions. The remainder of this section discusses how the original problem has shifted in light of the implemented solution.

The first and most important change is that the project has produced a working, fully reproducible pipeline that converts pre-execution commit metadata into a failure probability with a measured failure-class F1 of 0.42 under commit-grouped cross-validation and 0.40 on a per-repository chronological partition. These numbers represent a meaningful capability that did not exist at the start of the project. An organization that adopted the trained classifier could use it to route high-risk commits to a smaller pre-flight test suite, to defer them to off-peak compute capacity, to require pre-merge approval, or simply to alert the author for a second look. None of these workflows would have been supported by the prior art reviewed in Chapter 3 without the use of post-execution telemetry, which fundamentally changes the deployment posture.

The second change concerns the empirical understanding of what makes a CI/CD build fail at commit time. The exploratory data analysis in Chapter 7, combined with the ablation study and feature-importance analysis, revealed that the dominant predictive signals are structural rather than textual: the identity of the repository, the trigger event, the size of the commit, and the bot-versus-human authorship of the commit collectively account for the bulk of the model's discriminative ability. The textual signal from commit messages, while non-zero, is substantially weaker than the project's initial hypothesis assumed. This empirical finding is itself a contribution: it tells future researchers and practitioners that effort spent on richer text representations may have diminishing returns relative to effort spent on better-engineered structural features, particularly for the pre-execution prediction setting.

The third change is methodological rather than empirical, and it is the change the author regards as most durable. The project began by reporting a failure-class F1 of 0.5924 and now reports 0.4216 for the same classifier on the same data. The difference is not a deterioration in the system but a correction to the instrument used

to measure it, and Section 7.4.5 attributes it in full: 0.060 to commits shared between the training and test partitions, and 0.126 to selecting the decision threshold on the data used to report the result. The lesson generalises beyond this project. Both defects inflate a result while leaving every individual step of the analysis looking correct, neither is detectable from the reported metrics alone, and the second, which is the less conspicuous of the two, was worth more than twice the first.

The fourth change, in the spirit of intellectual honesty, is that the hybrid claim of the project—that combining textual and structured features outperforms either modality alone—was empirically refuted on this dataset at the default decision threshold. The structured-only ablation outperforms the full hybrid configuration on failure-class F1 (37.93 percent versus 32.21 percent) at the default threshold. While the gap narrows after threshold optimization and the hybrid model retains an edge on threshold-independent ranking metrics (ROC-AUC and PR-AUC), the unconditional version of the hybrid claim is not supported by this dataset. Section 8.3 discusses the implications of this finding in detail.

8.2 Benefits Achieved

Several concrete benefits have been achieved as a consequence of the implemented solution. These are organized by stakeholder perspective.

Benefits to research and academic readers. The project contributes a fully reproducible end-to-end pipeline that converts a real, freshly collected GitHub Actions dataset into a thoroughly evaluated hybrid machine learning system. The pipeline includes data collection, exploratory analysis, data preparation with explicit defenses against identity leakage, hybrid model construction with four parallel preprocessing branches, comparative evaluation of three classifiers under two split regimes, an ablation study, a threshold-optimization experiment, and a business-impact analysis. The full code, data, trained models, intermediate results, and figures are committed to the project repository under a fixed random seed, allowing any reader to reproduce every reported number to four decimal places. This level of reproducibility exceeds the norm in published machine learning papers on this topic and provides a useful reference implementation for future work.

Benefits to practitioners. The trained XGBoost classifier at threshold 0.06 is a deployment-ready artefact: it is a single joblib file under three megabytes that can be loaded into a Python process and queried with sub-millisecond latency on commodity hardware. A practitioner with an existing CI/CD environment could integrate the predictor as a webhook on the commit-creation event, surfacing a failure probability that downstream tooling could consume. The system imposes no failure mode of its own (a missing or rejected prediction degrades gracefully to the existing CI/CD

behavior), which is a property that lowers the practical adoption threshold substantially.

Benefits to the organization adopting the system. Under the operational scenario documented in Section 7.4.6, the system is estimated to save approximately $244,000 per year net of false alarms, for a mid-sized organization operating one thousand pipeline executions per day. This estimate is order-of-magnitude rather than precise. It is also conditional on an assumption the project did not measure, namely that a false alarm costs $2.50, and the selected configuration ceases to deliver a positive return once that cost exceeds $20.41. The savings accrue from two mechanisms: failed builds that are predicted in advance can be deferred to off-peak resources or skipped entirely, and failed builds that are caught early reduce the context-switching cost that developers incur when discovering failures hours after the originating commit.

Benefits to the student personally. The project required the integration of multiple disciplines (software engineering, data engineering, applied machine learning, technical writing) into a single cohesive deliverable, and surfaced a number of methodological subtleties (the post-execution feature boundary, identity leakage in TF-IDF, the importance of dual-split evaluation, the dominance of threshold calibration over feature engineering for imbalanced binary tasks) that would not have been visible from any narrower undertaking. The honest reporting of negative results (the refutation of the unconditional hybrid claim) was itself an exercise in academic discipline that the project has benefited from.

8.3 Remaining Limitations

Despite the accomplishments described above, several limitations remain that should be transparently acknowledged.

Limitation 1 — The hybrid claim is empirically weaker than the project's framing originally suggested. The ablation study in Section 7.4.3 demonstrates that the structured-only XGBoost configuration outperforms the full hybrid configuration on failure-class F1 at the default threshold. While the hybrid configuration retains a marginal edge on threshold-independent ranking metrics, and while the threshold-optimized hybrid configuration is the project's winning model overall, the unconditional claim that textual features add measurable predictive value beyond what structured features alone can provide is not supported by the data. Honest interpretation: on this dataset, with this text preprocessing, with this TF-IDF representation, the textual modality contributes weakly and inconsistently. Different datasets, richer text representations (transformer embeddings, for example), or stricter project-identity decontamination might restore the unconditional hybrid claim, but the present results do not.

Limitation 2 — Residual project-identity leakage in the textual features. The 693-token stoplist successfully eliminated the most obvious forms of author and project identity leakage (the words "anna", "kamat", "yagiz" that dominated the pre-cleaning vocabulary are gone), but the discriminative vocabulary after cleaning still contains tokens that are recognizably specific to particular repositories ("ractors" for Ruby, "callcache" for Rails, "ifrt" for JAX, "promql" for Prometheus). These tokens are genuine commit content rather than identifiers, so removing them by extending the stoplist would be over-aggressive and would damage the model. However, they do still allow the TF-IDF features to encode project membership indirectly, which means that the small predictive contribution attributable to the text branch is not entirely disentangled from the strong contribution of the repository categorical feature. A cleaner separation would require either character-level features (which would lose the unit of semantic interest) or a representation that is explicitly invariant to project-specific vocabulary.

Limitation 3 — The dataset is modest in size. The 9,772-row dataset is large enough to support credible statistical inference but small relative to industrial-scale build histories that may contain millions of records. The reported metrics should be expected to vary by a few percentage points on substantially larger or substantially different datasets. The architectural design of the project is fully compatible with larger datasets at no additional engineering cost, but the present numbers should not be quoted as universal benchmarks without acknowledging this sample-size caveat.

Limitation 4 — Open-source-only data. All 9,772 collected workflow runs come from public open-source repositories on GitHub. Commit conventions, failure-mode distributions, and even the meaning of categorical features such as branch names differ between open-source and corporate proprietary settings. The trained model's transfer behavior to a corporate dataset is unverified and should not be assumed to match its open-source performance.

Limitation 5 — The model is not currently deployed. The project produces a trained, evaluated, and serializable model, but does not include a live API endpoint, a CI/CD integration shim, or a monitoring dashboard for production use. Each of these would be a non-trivial additional engineering exercise and is identified as future work in Chapter 9.

Limitation 6 — The text representation is a baseline. The choice of TF-IDF with unigrams and bigrams is deliberate and defensible for an academic baseline, but it is also the simplest text representation that the project could have used. Transformer-based encoders such as Sentence-BERT or domain-pretrained CodeBERT might produce a richer textual signal that restores the unconditional hybrid claim. The future work section identifies this as a high-priority extension.

Limitation 7 — The collected workflow runs are not equally substantive. Every row in the dataset is one GitHub Actions workflow run, but the runs differ greatly in what they actually do, and the collector did not distinguish between them. The extreme case is elastic/elasticsearch, which contributes 600 runs and not one failure. Of those 600 runs, 591, or 98 percent, are documentation publishing jobs (docs-deploy, docs-build, docs-preview-cleanup) together with a Gradle wrapper checksum verification, and none of these compiles the project or executes a test suite. That repository exposes only six distinct workflows through the GitHub Actions API, against 137 for nestjs/nest and 107 for tensorflow/tensorflow, because its substantive test suite runs on Buildkite rather than on GitHub Actions. The 600-run-per-repository cap compounds the effect: at that level of activity the cap corresponds to a collection window of 13.8 hours, so the sample for that repository is narrow in workflow type and narrow in time simultaneously. The consequence is that the zero failure rate reported for elastic/elasticsearch in Figure 7.2 is a property of which workflows are observable through the API rather than a property of the project's engineering quality, and the same distortion applies in weaker form wherever a repository's peripheral automation outnumbers its build and test workflows. This does not invalidate the categorical-only result of Section 7.4.3, because workflow_name is itself one of the four categorical features and the association between the identity of a workflow and its outcome is genuine and available before execution. It does mean that part of what the categorical branch learns is the mundane fact that documentation jobs do not fail, which is a weaker finding than the prediction of a test-suite failure from the properties of a commit. A stratified collection design that sampled a fixed number of runs per workflow, or that filtered to the workflows which build or test the project, would remove the distortion and is identified as future work in Chapter 9.

8.4 Lessons Learned

The project surfaced a number of lessons that have value beyond the specific technical work and that the student carries forward into subsequent applied machine learning work.

Lesson 1 — Data quality is the rate-limiting step, and a credible study cannot be built on synthetic noise. The initial attempt to use a publicly available synthetic CI/CD failure logs dataset wasted approximately two days of effort before exploratory analysis revealed that the dataset's columns were statistically independent of one another and that the textual error messages were random character strings. No amount of clever modeling can compensate for the absence of signal in the underlying data. The pivot to collecting a real GitHub Actions dataset cost an additional two hours of engineering but produced a credible foundation for the entire downstream work.

Lesson 2 — Identity leakage in text features is real and requires aggressive intervention. Without aggressive text preprocessing and an algorithmically constructed stoplist, the TF-IDF representation learned to recognize author and project identifiers rather than commit content. The lesson generalizes: any model that consumes textual or identifier-like features should be tested for identity leakage by inspecting its discriminative vocabulary, and any token that is recognizable as an identifier rather than as content should be added to the cleaning stoplist.

Lesson 3 — The default 0.5 decision threshold is a trap under class imbalance. The default threshold is implicitly calibrated for a balanced class prior and is dramatically miscalibrated for the 8:1 ratio in this dataset, particularly for XGBoost. Threshold optimization should be treated as a first-class evaluation step, not as a footnote, and the resulting threshold should be reported alongside the metrics it produces.

Lesson 4 — The unit of observation must match the unit of feature construction. Every feature in this project describes a commit, while every row describes a workflow execution, and there are 3.45 executions per commit. A partition drawn over rows therefore placed near-duplicates of the same commit on both sides, and 88.3 percent of the test rows in the original design shared a commit with the training set. The defect is invisible in the reported metrics, survives every conventional check for overfitting, and inflated the headline figure by 0.060 of F1. Before any partition is drawn, the question that must be asked is what a row represents and whether two rows can be near-copies of one another; where they can, the partition must be grouped on whatever they are copies of.

Lesson 5 — A quantity selected to maximise a metric cannot also estimate it. The decision threshold in the original design was chosen by maximising failure-class F1 over the test set, and the resulting F1 was then reported as the model's performance on that test set. This is a circular procedure and it was worth 0.126 of F1, more than twice the leakage. It is also the easier of the two mistakes to make, because the threshold sweep is a legitimate technique and only its placement relative to the evaluation data is wrong. Any quantity tuned against data must be tuned on a partition that is then set aside.

Lesson 6 — Negative results are still results. The empirical refutation of the unconditional hybrid claim was not the outcome the project hoped for, but it is a contribution: it tells future researchers and practitioners that on real GitHub Actions data, basic TF-IDF on commit messages does not by itself produce a competitive failure classifier, and that effort should be redirected toward either richer text representations or better structured features. Hiding or rationalizing this result would have been intellectually dishonest and would have damaged the project's credibility on closer inspection.

Lesson 7 — Reproducibility is achievable and worth the effort. The discipline of pinning every dependency, fixing every random seed, committing every intermediate artefact, and writing orchestrator scripts that can regenerate every reported number from a clean checkout is not free, but it is far less expensive than the alternative of carrying every detail of the experiment in the student's memory and being unable to reconstruct it three months later. Future projects should adopt this discipline by default.

8.5 Comparison Before vs After

The contrast between the operational situation before the proposed solution and after its adoption can be summarized along several quantifiable dimensions, as presented in Table 8.1.

Table 8.1 — Operational comparison before and after the proposed solution.

Criterion	Before (Current Practice)	After (Proposed Solution)	Comment
When failures are detected	After full pipeline execution (minutes to hours after commit)	At commit time (sub-millisecond after commit)	The temporal shift is the central operational change.
Compute resources consumed before failure is known	Full build, test, and partial deploy stages	None	The pre-execution prediction allows resources to be conserved.
Developer context-switching cost	Average 15 minutes per failed build (literature estimate)	Reduced when high-risk commits are caught early	Hard to quantify precisely but consistent with cost-savings estimate.
Decision support for build prioritization	None (all builds run identically)	Per-commit failure probability available	Enables differentiated routing to fast/slow test suites.
Annual cost (assumed scale: 1,000 builds/day, 11% failure rate)	Baseline	Approximately $244,000 saved, net of false alarms	See Section 7.4.6 for assumptions and for the break-even false-alarm cost.
Visibility into per-repository failure rates	Limited (requires manual aggregation)	Quantified across the 18-repository sample	Surfaced as part of the EDA phase.
Reproducibility of evaluation	Variable across organizations	Fully reproducible from public dataset and code	Enables independent verification.
Feature-importance interpretability	Implicit	Top-30 feature importance available per model	Supports targeted CI/CD process improvements.

9. Conclusion and Future Work

9.1 Conclusion

This thesis has presented a Hybrid Machine Learning Pipeline for the prediction of CI/CD pipeline build failures at the moment of commit creation, using only pre-execution features that are available from the GitHub Actions REST API. The project began from a clear practical concern: software organizations spend significant compute resources and developer attention on CI/CD pipelines that fail eleven percent of the time, and existing predictive approaches in the academic literature rely on post-execution telemetry that cannot recover those resources. The project asked whether useful prediction was possible from commit-time information alone, and answered the question affirmatively through a complete and reproducible empirical study.

The principal results are as follows. Three machine learning classifiers (Logistic Regression, Random Forest, XGBoost) were trained and evaluated on a freshly collected dataset of 9,772 real workflow runs from eighteen popular open-source repositories on GitHub. Under commit-grouped five-fold cross-validation with the decision threshold selected outside the evaluation data, the three classifiers are not separable on failure-class F1. XGBoost is selected on PR-AUC and achieves a failure-class F1 of 0.4216 with a standard deviation across folds of 0.0638, a ROC-AUC of 0.824 and a PR-AUC of 0.480. The classifier is fully serializable, executes in sub-millisecond latency on commodity hardware, and is suitable for deployment as an advisory layer over existing CI/CD infrastructure. Under a documented operational scenario it is estimated to save approximately $244,000 per year net of false alarms, an estimate that ceases to be positive if a false alarm costs more than $20.41.

Beyond these headline results, the project surfaced three methodological findings that have value independent of the specific classifier. The first is the outcome of the ablation study. A configuration given nothing but categorical identity, namely the repository, the workflow name, the branch and the triggering event, outperforms the full hybrid model on every metric reported, at 0.4808 failure-class F1 against 0.4216 and 0.5467 PR-AUC against 0.4803. The hybrid hypothesis is therefore refuted more comprehensively than an earlier ablation indicated, and the honest characterisation of the system is that it is substantially a project-level risk estimator rather than a commit-level one.

The second is the attribution reported in Section 7.4.5. An earlier version of this work reported a failure-class F1 of 0.5924 under a protocol that shared commits between its training and test partitions and that selected the decision threshold on the test set it then reported. Correcting each defect independently shows that the

first was worth 0.060 and the second 0.126. That the less conspicuous defect was worth more than twice the conspicuous one is the finding the author considers most transferable.

The third concerns the stability of single-partition estimates. Because repository identity dominates the model and no grouped splitter balances repository composition, single-partition estimates of failure-class F1 on this dataset varied across a range of approximately 20 percentage points depending on which partition was drawn. Cross-validation is therefore not a refinement in this setting but a precondition for reporting a figure at all.

The project also produced a set of methodological artefacts that have value beyond their use in this thesis. The aggressive text-cleaning pipeline with its algorithmically constructed 693-token stoplist offers a transferable defense against identity leakage in TF-IDF-based software engineering text models. The four-branch ColumnTransformer architecture provides a clean reference implementation of early-fusion multimodal classification that can be adapted to other tabular-plus-text tasks. The fixed-seed reproducibility discipline applied throughout the codebase serves as a worked example of how to build an academic machine learning project that can be regenerated bit-for-bit by an independent reader.

The project did not produce every result it hoped for. The unconditional version of the hybrid claim—that combining textual and structured features outperforms either modality alone—was empirically refuted at the default decision threshold and only marginally supported at the optimized threshold. This negative result is treated by the thesis as a contribution rather than as a failure: it informs future researchers that on real GitHub Actions data with basic TF-IDF text features, the marginal value of the textual modality is smaller than the prior literature might suggest, and it redirects future effort either toward richer text representations or toward better-engineered structural features.

Taken together, the contributions of this project are: an empirical demonstration that pre-execution CI/CD failure prediction is feasible with credible accuracy on real data; a fully reproducible implementation of the proposed system together with all artefacts needed to verify its claims; a critical reassessment of the unconditional hybrid claim that runs through the prior literature; and a methodological reminder that for imbalanced binary classification, threshold calibration deserves first-class evaluation attention rather than relegation to a deployment afterthought.

9.2 Contributions

The specific contributions of the project, listed in order of decreasing importance, are as follows.

Contribution 1. A fully reproducible end-to-end pipeline for CI/CD failure prediction from pre-execution features, including data collection, exploratory analysis, feature engineering with identity-leakage defenses, hybrid model construction, comparative evaluation, ablation study, threshold optimization, and business-impact analysis. The pipeline is committed to a project repository under a fixed random seed, allowing every reported metric to be regenerated bit-for-bit by an independent reader.

Contribution 2. An empirical characterisation of what is and is not achievable in the strictly pre-execution setting on real GitHub Actions data. Under a commit-grouped protocol with the decision threshold selected outside the evaluation data, failure-class F1 reaches 0.4216 for the full hybrid model and 0.4808 for a model given only categorical identity. The second figure is the more informative of the two, because it establishes that the achievable performance in this setting derives principally from project-level base rates rather than from the content of individual commits. The author makes no claim to the strongest published result; the claim is that the figure is accompanied by a protocol sufficient to interpret it.

Contribution 3. A quantified case study in two evaluation defects that inflate reported performance in applied machine learning while leaving each individual step of the analysis apparently correct: partitioning at a finer grain than the features are constructed at, and selecting a decision threshold on the data used to report the result. Both are documented here with the magnitude of each isolated under otherwise identical conditions, at 0.060 and 0.126 of failure-class F1 respectively, together with the corrected protocol and the verification artefacts that demonstrate the correction.

Contribution 4. A critical empirical reassessment of the hybrid claim that runs through the CI/CD failure prediction literature, showing that on this dataset with basic TF-IDF text features, the textual modality adds at best marginal value beyond the structured features. The finding redirects future work toward richer text representations.

Contribution 5. A transferable identity-leakage defense for TF-IDF-based text models in software engineering domains, in the form of an algorithmically constructed stoplist that combines observed author logins, repository names, and bot signatures with manually curated noise tokens.

Contribution 6. A fresh public dataset of 9,772 real GitHub Actions workflow runs from eighteen open-source repositories, with full collection script and resulting CSV committed to the project repository for use as a benchmark by future researchers.

9.3 Future Work

Several natural extensions of the project would advance the research agenda materially.

Extension 1 — Transformer-based text encoders. Replacing the TF-IDF text branch with a transformer-based encoder (Sentence-BERT, CodeBERT, or a small domain-pretrained model) is the highest-priority extension. The ablation study suggests that the textual signal extracted by TF-IDF is too weak to drive a competitive failure classifier in isolation, but a richer representation that captures semantic similarity rather than surface-form lexical overlap might restore the unconditional hybrid claim. The trade-off would be the introduction of GPU compute as a deployment requirement and a loss of interpretability relative to the current TF-IDF coefficients.

Extension 2 — Larger and more diverse dataset. The 9,772-row dataset is sufficient for credible statistical inference but is small relative to the industrial-scale build histories that the system would ultimately be deployed against. Re-running the data collection at substantially larger scale (potentially one million workflow runs across hundreds of repositories) would test the generalizability of the reported metrics and would surface effects that are invisible at the present sample size.

Extension 3 — Corporate proprietary dataset validation. All evaluation in this project is performed on public open-source repositories. Corporate proprietary CI/CD environments may differ substantially in their commit conventions, failure-mode distributions, and operational scale. A follow-up study on a corporate dataset (under appropriate data-use agreements) would establish whether the trained model transfers across this boundary.

Extension 4 — Online learning and concept drift handling. The present model is trained once and applied to all subsequent commits. A production deployment would benefit from an online-learning extension that incrementally updates the model as new build outcomes are observed, both to track gradual concept drift and to adapt to new repositories, new workflows, or new failure modes that did not appear in the training window.

Extension 5 — Operationalization as a webhook service. Converting the trained pipeline into a live HTTP service that responds to GitHub webhooks at commit creation would close the gap between the academic deliverable and operational deployment. The webhook service would expose a single endpoint accepting a commit metadata payload and returning a failure probability and a recommended action; downstream tooling could consume this output to drive differentiated routing decisions.

Extension 6 — Per-repository fine-tuning. The current model is trained once across all eighteen repositories in the dataset. A per-repository fine-tuning approach

(or a multi-task learning approach that shares a backbone across repositories while learning repository-specific heads) might produce more accurate predictions on each individual repository, at the cost of additional training complexity and the need for sufficient data per repository.

Extension 7 — Interpretability and explanation generation. The trained XGBoost classifier produces feature-importance scores that indicate which features drive its predictions globally, but it does not produce per-commit explanations that a developer could read to understand why a specific commit was flagged. Integrating SHAP values or counterfactual explanations would close this gap and would substantially improve the human-machine interaction surface of the deployed system.

Extension 8 — Cost-sensitive evaluation. The present project optimizes the F1 metric and produces a business-impact estimate after the fact. A more rigorous approach would optimize directly for the business cost function (the cost of false alarms versus the cost of missed failures), choosing the decision threshold to minimize that cost rather than to maximize a statistical metric. The threshold-optimization module already exposes this option; the future work would consist of validating it under realistic cost parameters elicited from operational stakeholders.

Extension 9 — Stratified collection by workflow type. The present collector takes the most recent runs per repository up to a fixed cap, which allows a repository's peripheral automation to dominate its sample, as Limitation 7 in Section 8.3 records for elastic/elasticsearch. A stratified design that sampled a fixed number of runs per workflow, or that restricted collection to the workflows which build or test the project, would produce a corpus in which every row represents comparable work and would allow the per-repository failure rates to be read as a property of the projects rather than of the collection procedure.

Appendices

Appendix A: Source Code Listings

The complete source code of the project is available in the project repository. The principal modules are listed below with their line counts and primary responsibilities.

Module	Lines of Code	Primary Responsibility
src/collect_github_data.py	285	Data collection from GitHub Actions API
src/eda.py	330	Exploratory data analysis and chart generation
src/data_preparation.py	442	Cleaning, feature engineering, splits
src/hybrid_pipeline.py	281	Four-branch ColumnTransformer + classifiers
src/train_evaluate.py	415	Full training, evaluation, ablation study
src/threshold_optimization.py	178	Decision-threshold sweep
src/visualization.py	156	ThesisPlotter utility class
Total	**2,087**	

Appendix B: Reproduction Instructions

Every figure and every metric reported in this thesis is regenerated from the committed raw dataset by the commands below. An earlier version of this appendix referred to two scripts, `src/run_phase0.py` and `src/run_phase1.py`, that do not exist in the repository, and specified an ordering that could not have completed, because `src/run_phase2.py` imported two symbols that the Phase 2.5 module had removed and therefore failed on load. Both defects are corrected, and the sequence below has been executed from a clean checkout.

1. Clone the repository and create the environment.

``bash git clone <project-repository-url> cd cicd-failure-prediction python3.11 -m venv .venv .venv/bin/pip install -r requirements.txt ``

The library versions are pinned in `requirements.txt`. The results in this thesis were produced with scikit-learn 1.8.0 and xgboost 3.2.0; other versions of those two packages are not guaranteed to reproduce the figures exactly.

2. Reproduce every reported result.

``bash ./reproduce.sh ``

This is the only command required. It runs data preparation, constructs the commit-grouped and per-repository chronological partitions, verifies their integrity, executes the cross-validated evaluation, computes the business analysis, and renders the figures. It takes approximately three minutes on commodity hardware and writes

to `results/` and `figures/`.

The equivalent steps, if run individually:

``bash .venv/bin/python -m src.run_phase2_5 # prepare data, build and verify all splits .venv/bin/python -m src.run_corrected_evaluation # cross-validated evaluation, business model, figures ``

3. Optional steps.

```bash
```

Re-collect the dataset from the GitHub Actions API. Approximately two hours

because of API rate limits. The collected dataset is committed to data/raw/,

so this is not required.

export GITHUB_TOKEN="your_personal_access_token" .venv/bin/python -m src.collect_github_data

Exploratory data analysis figures.

.venv/bin/python -m src.run_phase2

Pipeline architecture validation.

.venv/bin/python -m src.run_phase3

Train and persist the deployable model artefacts to models/. This evaluates

on a single held-out fold and writes to results/single_fold_reference/;

those figures are not the reported results. See results/README.md.

.venv/bin/python -m src.run_phase4 ```

4. Verifying the results rather than trusting them.

`results/README.md` states which file is authoritative for each reported number. Three files exist specifically so that the claims of this thesis can be checked rather than accepted: `results/split_integrity.json` records the commit overlap between every pair of partitions and the temporal invariant for each of the eighteen repositories; `results/metric_attribution_ladder.json` records the decomposition presented in Section 7.4.5; and `results/threshold_selection_gap.json` records the residual optimism discussed in Section 7.4.4.

All outputs are deterministic given the fixed random seed of 42.

Appendix C: Complete Metrics Tables

Table C.1 reports every classifier under both evaluation protocols at both the default and the selected decision threshold. Metrics under the commit-grouped protocol are computed over pooled out-of-fold predictions, so each of the 9,772 workflow runs contributes exactly one prediction from a model trained without any run of its commit. Metrics under the chronological protocol are computed on the 1,636 runs of the per-repository chronological test partition, from models trained on that partition's own training fold.

Table C.1 — Complete classifier metrics (all conditions).

Model	Protocol	Threshold	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	Commit-grouped CV	0.50 (default)	76.29%	74.09%	27.55%	71.27%	39.74%	82.76%	40.92%
Logistic Regression	Commit-grouped CV	0.646 (selected)	83.39%	71.53%	34.34%	56.34%	42.67%	82.76%	40.92%
Logistic Regression	Chronological	0.50	87.16%	67.02%	34.95%	42.21%	38.24%	81.62%	36.49%
Logistic Regression	Chronological	0.690 (selected)	87.16%	67.02%	34.95%	42.21%	38.24%	81.62%	36.49%
Random Forest	Commit-grouped CV	0.50	83.01%	70.79%	33.39%	55.13%	41.59%	80.08%	39.09%
Random Forest	Commit-grouped CV	0.538 (selected)	86.70%	68.93%	40.64%	46.18%	43.23%	80.08%	39.09%
Random Forest	Chronological	0.510 (selected)	86.25%	68.55%	33.49%	46.75%	39.02%	82.18%	38.17%
XGBoost	Commit-grouped CV	0.50	90.36%	59.17%	73.05%	19.22%	30.43%	82.40%	48.03%
XGBoost	**Commit-grouped CV**	**0.076 (selected)**	**89.71%**	**66.86%**	**54.46%**	**37.59%**	**44.48%**	**82.40%**	**48.03%**

Model	Protocol	Threshold	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost	Chronological	0.050 (selected)	89.30%	66.17%	42.34%	37.66%	39.86%	80.27%	41.81%

The selected configuration, XGBoost under the commit-grouped protocol, is highlighted in bold. It is selected on PR-AUC rather than on F1, because the failure-class F1 scores of the three classifiers are not separable at this sample size, as Table 7.2 establishes.

Table C.2 reports the failure-class F1 obtained in each individual fold, which is the basis for the claim that the classifiers cannot be separated and that single-partition estimates on this dataset should not be relied upon.

Table C.2 — Failure-class F1 by fold.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std. dev.
Logistic Regression	0.5380	0.4366	0.3788	0.3983	0.4038	0.4311	0.0633
Random Forest	0.5233	0.3739	0.4829	0.3213	0.4188	0.4240	0.0812
XGBoost	0.4309	0.4200	0.4976	0.3208	0.4386	0.4216	0.0638

Table C.3 reports the ablation configurations under the same protocol.

Table C.3 — Feature-set ablation, commit-grouped cross-validation, XGBoost throughout.

Configuration	F1 (mean)	F1 (std. dev.)	Precision	Recall	ROC-AUC	PR-AUC
Text only	0.1962	0.0661	0.3131	0.1559	0.6654	0.2080
Hybrid (all four branches)	0.4216	0.0638	0.5204	0.3686	0.8240	0.4803
Structured only	0.4225	0.0655	0.5380	0.3611	0.8197	0.4825
Categorical only	0.4808	0.0767	0.5911	0.4319	0.8649	0.5467

Every figure in this appendix is generated by `src/run_corrected_evaluation.py` and written to `results/thesis_tables.json`, from which these tables are transcribed. Split integrity is recorded separately in `results/split_integrity.json` and the attribution of Section 7.4.5 in `results/metric_attribution_ladder.json`.

Appendix D: Hyperparameter Configurations

The complete hyperparameter configurations for the three classifiers are listed in Table D.1.

Table D.1 — Classifier hyperparameter configurations.

Hyperparameter	Logistic Regression	Random Forest	XGBoost
solver	liblinear	n/a	n/a
n_estimators	n/a	200	300

Hyperparameter	Logistic Regression	Random Forest	XGBoost
max_depth	n/a	25	8
max_iter	1000	n/a	n/a
min_samples_split	n/a	10	n/a
min_samples_leaf	n/a	4	n/a
learning_rate	n/a	n/a	0.1
subsample	n/a	n/a	0.85
colsample_bytree	n/a	n/a	0.85
reg_alpha (L1)	n/a	n/a	0.1
reg_lambda (L2)	n/a	n/a	1.0
class_weight	balanced	balanced	n/a
scale_pos_weight	n/a	n/a	8.12
eval_metric	n/a	n/a	logloss
tree_method	n/a	n/a	hist
random_state	42	42	42
n_jobs	n/a	-1	-1

Appendix E: Final Submission Checklist

The following checklist was completed before final submission of this thesis.

- [x] Cover page completed with university details, student information, and submission date
- [x] All template instructions and placeholders removed or replaced (except supervisor name pending)
- [x] Problem statement clearly aligned with the four stated objectives
- [x] Existing solution approaches reviewed and compared in tabular form
- [x] Proposed solution justified with explicit rationale and architectural diagram
- [x] Implementation documented at module-by-module level
- [x] Testing methodology defined, validation procedures executed, results reported
- [x] Discussion addresses both achievements and remaining limitations honestly
- [x] Conclusion summarizes contributions and identifies eight specific future-work items
- [x] References listed in IEEE format with twenty academic sources
- [x] Appendices include source code summary, reproduction instructions, complete metrics, hyperparameters, and submission checklist
- [x] Document language, formatting, headings, and figures consistent throughout

References

[1] A. Patel, "Research the Use of Machine Learning Models to Predict and Prevent Failures in CI/CD Pipelines and Infrastructure," International Journal of Engineering Research & Technology, vol. 8, no. 11, 2019.

[2] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research on Continuous Integration," in Proceedings of the 14th International Conference on Mining Software Repositories, 2017, pp. 447–450. doi: 10.1109/MSR.2017.24.

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[6] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.

[7] D. M. W. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," Journal of Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.

[8] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in Proceedings of the 23rd International Conference on Machine Learning, 2006, pp. 233–240. doi: 10.1145/1143844.1143874.

[9] H. He and E. A. Garcia, "Learning from Imbalanced Data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

[10] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[11] M. Fowler, "Continuous Integration," martinfowler.com, 2006. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html

[12] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[13] A. E. Hassan and R. C. Holt, "The Top Ten List: Dynamic Fault Prediction," in Proceedings of the 21st IEEE International Conference on Software Maintenance, 2005, pp. 263–272. doi: 10.1109/ICSM.2005.91.

[14] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting Defects for Eclipse," in Proceedings of the 3rd International Workshop on Predictor Models in Software Engineering, 2007, pp. 9–15.

[15] S. Kim, T. Zimmermann, K. Pan, and E. J. Whitehead, "Automatic Identification of Bug-Introducing Changes," in Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering, 2006, pp. 81–90.

[16] J. Eyrolle and J.-M. Cellier, "The Effects of Interruptions in Work Activity: Field and Laboratory Results," Applied Ergonomics, vol. 31, no. 5, pp. 537–543, 2000.

[17] GitHub, Inc., "GitHub Actions API Reference," GitHub Docs. [Online]. Available: https://docs.github.com/en/rest/actions

[18] W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[19] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55.

[20] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[21] S. Kapoor and A. Narayanan, "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science," Patterns, vol. 4, no. 9, art. 100804, 2023. doi: 10.1016/j.patter.2023.100804.

[22] D. R. Roberts et al., "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure," Ecography, vol. 40, no. 8, pp. 913–929, 2017. doi: 10.1111/ecog.02881.

[23] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLOS ONE, vol. 10, no. 3, art. e0118432, 2015. doi: 10.1371/journal.pone.0118432.

[24] A. E. Hassan and K. Zhang, "Using Decision Trees to Predict the Certification Result of a Build," in Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering, 2006.

[25] L. Madeyski and M. Kawalerowicz, "Continuous Defect Prediction: The Idea and a Related Dataset," in Proceedings of the 14th International Conference on Mining Software Repositories, Buenos Aires, Argentina, 2017, pp. 515–518. doi: 10.1109/MSR.2017.46.

[26] R. Dhawan and M. Dhawan, "AI-Augmented Reliability in CI/CD: A Framework for Predictive, Adaptive, and Self-Correcting Pipelines," Frontiers in Artificial Intelligence, vol. 9, art. 1776546, 2026. doi: 10.3389/frai.2026.1776546.

[27] R. Sharma, E. Petrova, and J. O. Connolly, "Machine Learning-Based Failure Prediction in Continuous Integration and Deployment Workflows," unpublished manuscript, Nov. 12, 2025. [Online]. Available: https://www.researchgate.net/publication/401540054 (accessed Sep. 7, 2026).